

B 不确定性与拒识能力分析（补充实验）

本节为问题二的可选补充实验，不属于题目硬性要求，正文仅保留结论。

模型在式 (11) 中已输出预测方差，据此可定义两类不确定度。偶然不确定度取对数方差头输出的指数：

$$\sigma_{\text{model}} = \exp\left(\frac{\ln \sigma^2}{2}\right), \quad (33)$$

认知不确定度由集成成员的分歧给出：

$$\sigma_{\text{ens}} = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (\mu_k - \bar{\mu})^2}, \quad \sigma_{\text{total}} = \sqrt{\sigma_{\text{model}}^2 + \sigma_{\text{ens}}^2}. \quad (34)$$

另一类与极性判别相关的不确定度取分类头概率熵：

$$H = - \sum_{k=0}^2 \pi_k \ln \pi_k. \quad (35)$$

按不确定度升序保留前 q 比例样本，可得风险—覆盖率曲线，其积分即 AURC[7]：

$$\text{AURC} = \int_0^1 \text{MAE}_q \, dq. \quad (36)$$

两类不确定度对分类准确率的影响方向并不一致，按总不确定度剔除会使准确率下降，按分类头概率熵剔除则明显上升。就强度回归而言，剔除最不确定的 10% 样本后平均绝对误差由 0.5866 降至 0.5465，剔除一半后降至 0.4495；风险—覆盖率曲线的曲线下面积（AURC）为 0.4365（随机剔除对应 0.5866、理论下界 0.2258）；总不确定度与绝对误差的斯皮尔曼相关系数为 0.296（验证集）与 0.230（测试划分），均为正。就极性判别而言，按总不确定度剔除的曲线下面积 0.3980 甚至劣于随机的 0.3777，改用分类头概率熵后降至 0.2252。相关曲线见 5.4.5 节的图 21 至图 23。

用单一不确定度同时拒识回归与分类，在本任务上并不成立，原因在于总不确定度与真值强度单调同增（五分位分桶后最确定一桶的真值绝对强度均值 0.411、最不确定一桶 1.233），实质追踪的是强度幅值；分类头概率熵则与强度递减，追踪的是极性模糊度。据此本文把两类不确定度分别用于强度拒识与极性拒识。

C 核心代码

本节给出问题二与问题三模型的核心代码，与提交的支撑材料中的 `code/` 目录为同一份；问题二的代码对应正文 5.3 节，问题三的代码对应正文第 6 章。代码以 Python 3.12 与 PyTorch 编写，运行顺序见 `code/README.md`。为便于与提交材料核对，每个代码块标注其在支撑材料中的相对路径与总行数。为避免中文字体下缺字形，代码中的少量 Unicode 数学符号（如 $\in$ 、 $\leq$ 、 $\rightarrow$ 、 π ）在附录中以 ASCII 等价形式给出（如 `in`、`<=`、`->`、`pi`），仅涉及注释、文档字符串与输出文案，不影响可执行语义；原始文件以提交的

code/ 目录为准。问题一的特征提取与时序对齐脚本及其复现说明随问题一材料一并提交（见表 22 备注）。

C.1 问题二：MRF-Net 网络结构与动态专家门控

code/02_model/model.py（共 285 行）

```
1  # -*- coding: utf-8 -*-
2  """
3  MRF-Net: Missing-Robust Fusion Network
4  掩码感知编码 -> 跨模态重建 -> 代理令牌门控 -> 共享/特有分解 -> 动态专家融合 -> 不确定性双头
5  """
6  import math
7
8  import torch
9  import torch.nn as nn
10 import torch.nn.functional as F
11
12 MODALITIES = ("text", "audio", "vision")
13
14
15 class PositionalEncoding(nn.Module):
16     def __init__(self, d, max_len=2048):
17         super().__init__()
18         pe = torch.zeros(max_len, d)
19         pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
20         n_even = pe[:, 0::2].shape[1]
21         div = torch.exp(torch.arange(0, n_even, dtype=torch.float) * (-math.log(10000.0)
22 / d))
23         pe[:, 0::2] = torch.sin(pos * div)
24         pe[:, 1::2] = torch.cos(pos * div[:, :pe[:, 1::2].shape[1]])
25         self.register_buffer("pe", pe.unsqueeze(0))
26
27     def forward(self, x):
28         return x + self.pe[:, :x.size(1)]
29
30 class ModalityEncoder(nn.Module):
31     """掩码感知编码器：显式把“这一段是瞎的”喂给网络。”"""
32
33     def __init__(self, d_in, hidden, nhead, layers, dropout, down=1):
34         super().__init__()
35         # 逐时间步线性投影（比大核卷积省 5 倍算力）+ 轻量局部混合
36         self.inp = nn.Sequential(nn.Linear(d_in + 2, hidden), nn.GELU())
37         self.local = nn.Conv1d(hidden, hidden, kernel_size=3, padding=1)
38         self.pos = PositionalEncoding(hidden)
39         layer = nn.TransformerEncoderLayer(
```

```

40         hidden, nhead, dim_feedforward=hidden * 4,
41         dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
42     self.enc = nn.TransformerEncoder(layer, num_layers=layers, enable_nested_tensor=
False)
43     self.down = None
44     if down > 1:
45         self.down = nn.Conv1d(hidden, hidden, kernel_size=down, stride=down)
46
47     def forward(self, x, miss, pad):
48         m = miss.float().unsqueeze(-1)          # 1=缺失（显式告知网络该段不可用）
49         v = 1.0 - m
50         h = self.inp(torch.cat([x * v, m, v], dim=-1))
51         h = h + F.gelu(self.local(h.transpose(1, 2))).transpose(1, 2)
52         h = self.enc(self.pos(h), src_key_padding_mask=pad)
53         return h                                # 原生序列长度
54
55     def downsample(self, h, miss, pad):
56         if self.down is None:
57             return h, miss, pad
58         h2 = self.down(h.transpose(1, 2)).transpose(1, 2)
59         k, s = self.down.kernel_size[0], self.down.stride[0]
60         md = F.max_pool1d(miss.unsqueeze(1).float(), k, s).squeeze(1).bool()
61         pd = F.max_pool1d(pad.unsqueeze(1).float(), k, s).squeeze(1).bool()
62         return h2, md, pd
63
64
65     class Expert(nn.Module):
66         """跨模态 + 跨时间自注意力专家。"""
67
68     def __init__(self, hidden, nhead, dropout):
69         super().__init__()
70         self.attn = nn.MultiheadAttention(hidden, nhead, dropout=dropout, batch_first=
True)
71         self.norm1 = nn.LayerNorm(hidden)
72         self.norm2 = nn.LayerNorm(hidden)
73         self.ffn = nn.Sequential(
74             nn.Linear(hidden, hidden * 2), nn.GELU(), nn.Dropout(dropout),
75             nn.Linear(hidden * 2, hidden))
76
77     def forward(self, tok, pad):
78         a, _ = self.attn(tok, tok, tok, key_padding_mask=pad)
79         x = self.norm1(tok + a)
80         x = self.norm2(x + self.ffn(x))
81         return x
82
83

```

```

84 class MRFNet(nn.Module):
85     def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
86                 dropout=0.1, downs=None, target_len=50, mag_levels=0,
87                 ms_pool=False, ms_windows=(5, 25, 50), use_pol=False):
88         super().__init__()
89         downs = downs or {"text": 1, "audio": 1, "vision": 1}
90         self.dims = dims
91         self.hidden = hidden
92         self.target_len = target_len
93         self.mag_levels = int(mag_levels)
94         self.ms_pool = bool(ms_pool)
95         self.ms_windows = tuple(int(w) for w in ms_windows)
96         self.use_pol = bool(use_pol)
97         self.mods = list(MODALITIES)
98
99         self.enc = nn.ModuleDict({
100             m: ModalityEncoder(dims[m], hidden, nhead, enc_layers, dropout, downs[m])
101             for m in self.mods})
102
103         # —— 跨模态重建（缺失位置靠“造”） ——
104         self.cross = nn.ModuleDict({m: nn.MultiheadAttention(hidden, nhead, dropout=
105 dropout, batch_first=True) for m in self.mods})
106         self.dec = nn.ModuleDict({m: nn.Linear(hidden, dims[m]) for m in self.mods})
107
108         # —— 代理令牌 ——
109         self.proxy = nn.ParameterDict({m: nn.Parameter(torch.randn(hidden) * 0.02) for m
110 in self.mods})
111         self.beta = nn.ModuleDict({m: nn.Linear(hidden + 1, 1) for m in self.mods})
112
113         # —— 共享 / 特有分解 ——
114         self.share = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
115         self.spec = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
116         self.mix = nn.ModuleDict({m: nn.Linear(hidden * 2, hidden) for m in self.mods})
117
118         # —— 动态专家融合 ——
119         self.mod_emb = nn.Embedding(3, hidden)
120         self.experts = nn.ModuleList([Expert(hidden, nhead, dropout) for _ in range(
121 n_experts)])
122         self.gate = nn.Sequential(nn.Linear(hidden + 7, hidden), nn.GELU(), nn.Linear(
123 hidden, n_experts))
124         self.pool_score = nn.Sequential(nn.Linear(hidden, hidden // 2), nn.Tanh(), nn.
125 Linear(hidden // 2, 1))
126
127         # —— 输出头 ——
128         self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
129 hidden, 1))

```

```

124         self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
125             hidden, 1))
126
127         self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
128             hidden, 3))
129
130     # —— 强度分级辅助头（CORAL 有序回归：K-1 个累计 logit） ——
131     # 动机（实测）：61.7% 的样本  $|y| \leq 2/3$ （弱标注）而 ACC 仅 0.486； $|y|=1/3$  层
132     # ACC 更低至 0.361 —— 模型分不清“完全中性”与“轻微极性”。该头把  $|y|$  离散为
133     # K 个等级（步长 1/3）做有序回归，强制模型在零点附近建立分辨率。
134     if self.mag_levels > 1:
135         self.mag_w = nn.Linear(hidden, 1, bias=False)
136         self.mag_b = nn.Parameter(torch.linspace(-1.5, 1.5, self.mag_levels - 1))
137
138     # —— 多尺度时间池化（补足单层注意力池化“过于均匀”的问题） ——
139     # 实测：帧注意力熵 3.86 / 上限 3.91  $\approx$  均匀，说明注意力没聚焦到关键片段。
140     # 改为“多窗口掩码均值池化 + 线性投影”，并与注意力池化残差相加。
141     if self.ms_pool:
142         T = 3 * int(target_len)          # 三模态拼接后的 token 长度
143         nseg = sum(max(1, T // w) for w in self.ms_windows)
144         self._ms_nseg = nseg
145         self._ms_T = T
146         self.ms_proj = nn.Linear(nseg * hidden, hidden)
147
148     # —— 二分类极性辅助头（对齐文献 Acc-2 的做法） ——
149     if self.use_pol:
150         self.head_pol = nn.Linear(hidden, 1)
151
152     # -----
153     def _masked_win_pool(self, h, pad):
154         """多窗口掩码均值池化：把时间轴切成若干段，段内对非填充位置求均值。
155
156         h:(B,T,H) pad:(B,T) bool(True=填充) 返回 (B, nseg*H)
157         """
158         h = h.masked_fill(pad.unsqueeze(-1), 0.0)
159         valid = (~pad).float().unsqueeze(-1)          # (B,T,1)
160         T = h.size(1)
161         outs = []
162         for w in self.ms_windows:
163             nb = max(1, T // w)
164             seg = torch.arange(T, device=h.device) * nb // T    # 把 T 均分到 nb 段
165             for b in range(nb):
166                 m = (seg == b).float().unsqueeze(0).unsqueeze(-1)    # (1,T,1)
167                 s = (h * m * valid).sum(dim=1)
168                 n = (m * valid).sum(dim=1).clamp(min=1.0)
169                 outs.append(s / n)
170         return torch.cat(outs, dim=-1)

```

```

168
169 @staticmethod
170 def _availability(rho):
171     """rho:(B,3) 缺失率 -> (B,7) 可用性特征。"""
172     n_vis = (rho < 0.999).float().sum(dim=1, keepdim=True)
173     return torch.cat([rho, (rho < 0.999).float(), n_vis / 3.0], dim=1)
174
175 def forward(self, x, miss, pad, return_aux=False):
176     """x/miss/pad: dict 模态 -> (B,L,d)/(B,L)/(B,L)"""
177     h, rec, share, spec = {}, {}, {}, {}
178     for m in self.mods:
179         h[m] = self.enc[m](x[m], miss[m], pad[m])
180
181     # --- 1) 跨模态重建 (原生长度上计算, 损失只取缺失位置) ---
182     for m in self.mods:
183         others = [h[o] for o in self.mods if o != m]
184         kv = torch.cat(others, dim=1)
185         kv_pad = torch.cat([pad[o] for o in self.mods if o != m], dim=1)
186         a, _ = self.cross[m](h[m], kv, kv, key_padding_mask=kv_pad)
187         h_hat = h[m] + a * miss[m].float().unsqueeze(-1)
188         rec[m] = self.dec[m](h_hat)
189
190     # --- 2) 降采样到 target_len 统一时间尺度 ---
191     hd, md, pd_ = {}, {}, {}
192     for m in self.mods:
193         hd[m], md[m], pd_[m] = self.enc[m].downsample(h[m], miss[m], pad[m])
194
195     # --- 3) 代理令牌门控 + 分解 ---
196     rho = torch.stack([md[m].float().mean(dim=1) for m in self.mods], dim=1) # (B,3)
197     beta_val = {}
198     for m in self.mods:
199         hm = hd[m]
200         inp = torch.cat([hm.mean(dim=1), rho[:, self.mods.index(m)].unsqueeze(-1)],
201 dim=-1)
202         b = torch.sigmoid(self.beta[m](inp)).unsqueeze(1) # (B,1,1)
203         beta_val[m] = b.detach().view(-1)
204         hm = (1 - b) * hm + b * self.proxy[m].view(1, 1, -1)
205         share[m] = self.share[m](hm)
206         spec[m] = self.spec[m](hm)
207         hd[m] = self.mix[m](torch.cat([share[m], spec[m]], dim=-1))
208
209     # --- 4) 动态专家融合 ---
210     tokens = torch.cat([hd[m] for m in self.mods], dim=1)
211     tok_pad = torch.cat([pd_[m] for m in self.mods], dim=1)
212     mid = torch.arange(3, device=tokens.device).repeat_interleave(tokens.size(1) //
3)

```

```

212     tokens = tokens + self.mod_emb(mid).unsqueeze(0)
213
214     av = self._availability(rho)
215     cls_pool = tokens.mean(dim=1)
216     g = torch.softmax(self.gate(torch.cat([cls_pool, av], dim=-1)), dim=-1)      # (B,E)
217
218     outs = torch.stack([e(tokens, tok_pad) for e in self.experts], dim=1)      # (B,E,
T,H)
219     fused = (g.unsqueeze(-1).unsqueeze(-1) * outs).sum(dim=1)                # (B,T,H
)
220
221     score = self.pool_score(fused).masked_fill(tok_pad.unsqueeze(-1), -1e9)
222     alpha = torch.softmax(score, dim=1)                                       # (B,T
,1)
223     z = (alpha * fused).sum(dim=1)
224     if self.ms_pool:
225         pooled = self._masked_win_pool(fused, tok_pad)
226         z = z + self.ms_proj(pooled)
227
228     mu = self.head_mu(z).squeeze(-1)
229     logvar = self.head_lv(z).squeeze(-1).clamp(-6.0, 4.0)
230     logits = self.head_cls(z)
231
232     if not return_aux:
233         return mu, logvar, logits
234     aux = dict(rec=rec, share=share, spec=spec, gate=g, alpha=alpha,
235               rho=rho, hd=hd, pad_ds=pd_, miss_ds=md, z=z, mu=mu, logits=logits,
236               beta=beta_val)
237     if self.mag_levels > 1:
238         aux["logits_mag"] = self.mag_w(z) + self.mag_b.view(1, -1)
239     if self.use_pol:
240         aux["pol"] = self.head_pol(z).squeeze(-1)
241     return mu, logvar, logits, aux
242
243
244 class PlainFusion(nn.Module):
245     """A0/A1 基线: **朴素拼接 + MLP** 融合（无掩码感知编码/重建/代理令牌/共享-特有分解/动态专家）。
246
247     用于论文消融中的“常规固定权重融合模型”对照：
248     * “use_indicator=False“ -> **A0**（完全不做缺失处理的常规模型）
249     * “use_indicator=True“ -> **A1**（仅额外告知“哪段缺失”）
250
251     接口与 :class:‘MRFNet‘ 完全一致（“forward“ 返回 “mu/logvar/logits[+aux]“），
252     因此可直接复用 train/runtime/infer 的全部代码。
253     """
254

```

```

255 def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
256             dropout=0.1, downs=None, target_len=50, use_indicator=True):
257     super().__init__()
258     self.mods = list(MODALITIES)
259     self.use_indicator = bool(use_indicator)
260     d_in = sum(int(dims[m]) for m in self.mods)
261     if self.use_indicator:
262         d_in += 2 * len(self.mods)          # 每模态: 缺失率 + 可见率
263     self.net = nn.Sequential(
264         nn.Linear(d_in, hidden), nn.GELU(), nn.Dropout(dropout),
265         nn.Linear(hidden, hidden), nn.GELU())
266     self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
267         hidden, 1))
268     self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
269         hidden, 1))
270     self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
271         hidden, 3))
272
273 def forward(self, x, miss, pad, return_aux=False):
274     feats = []
275     for m in self.mods:
276         v = (~pad[m]).float().unsqueeze(-1)          # (B,L,1)
277         pooled = (x[m] * v).sum(dim=1) / v.sum(dim=1).clamp(min=1.0)  # 有效位均值池化
278         feats.append(pooled)
279         if self.use_indicator:
280             mval = (miss[m] & ~pad[m]).float().mean(dim=1, keepdim=True)
281             feats += [mval, 1.0 - mval]
282     h = self.net(torch.cat(feats, dim=-1))
283     mu = self.head_mu(h).squeeze(-1)
284     logvar = self.head_lv(h).squeeze(-1).clamp(-6.0, 4.0)
285     logits = self.head_cls(h)
286     if not return_aux:
287         return mu, logvar, logits
288     return mu, logvar, logits, dict(z=h)

```

C.2 问题二：损失函数（异方差回归、类别加权、一致性、重建与蒸馏）

code/02_model/losses.py（共 375 行）

```

1  #-*- coding: utf-8 -*-
2  """损失函数与评价指标。"""
3  import numpy as np
4  import torch
5  import torch.nn.functional as F
6
7  MODALITIES = ("text", "audio", "vision")

```

```

8
9
10 # -----
11 # 损失
12 # -----
13 def reg_nll(mu, logvar, y):
14     """高斯负对数似然（仅作不确定性辅助项）。"""
15     return 0.5 * (torch.exp(-logvar) * (y - mu) ** 2 + logvar).mean()
16
17
18 def reg_main(mu, y, beta=0.5):
19     """主回归损失：SmoothL1，与官方评价指标 MAE 对齐。"""
20     return F.smooth_l1_loss(mu, y, beta=beta)
21
22
23 def cls_ce(logits, y, weight=None, label_smoothing=0.0):
24     return F.cross_entropy(logits, y, weight=weight, label_smoothing=label_smoothing)
25
26
27 # -----
28 # D 系列：针对“弱标注样本”（|y| 小 / 中性类）的标签分布与有序损失
29 # 实测（附件2）：|y| 以 1/3 为步长、最大 3.0，且 类 = sign(均值强度)。
30 # 因此 |y| 本身就是“标注一致性”的直接度量：|y|=1/3 相当于标注者 2:1 分裂。
31 # 三类混淆中 84% 只是邻类错，说明需要**序信息**而不是更多容量。
32 # -----
33 def intensity_soft_target(y_reg, tau=0.3):
34     """用连续强度 |y| 构造三分类**软目标**（标签分布 / 置信度感知平滑）。
35
36     q_极性 = tanh(|y|/tau), q_中性 = 1 - q_极性, y=0 时 q = 纯中性
37
38     tau 越小越接近硬标签；tau->inf 退化为“只区分中性/非中性”。
39     """
40     y = y_reg
41     conf = torch.tanh(torch.abs(y) / max(float(tau), 1e-6)).clamp(0.0, 1.0)
42     q = torch.zeros(y.shape[0], 3, dtype=y.dtype, device=y.device)
43     pos = y > 0
44     neg = y < 0
45     q[:, 1] = 1.0 - conf
46     q[pos, 2] = conf[pos]
47     q[neg, 0] = conf[neg]
48     q[~(pos | neg), 1] = 1.0
49     return q / q.sum(dim=-1, keepdim=True).clamp(min=1e-8)
50
51
52 def emd2_loss(logits, q):
53     """EMD^2（有序）惩罚：累计分布的平方差，使“跨类错误”代价 > “邻类错误”。"""

```

```

54     p = torch.softmax(logits, dim=-1)
55     return ((torch.cumsum(p, dim=-1) - torch.cumsum(q, dim=-1)) ** 2).sum(-1).mean()
56
57
58 def neu_suppress_loss(logits, y_cls, lam=0.0):
59     """中性抑制正则（代价敏感学习）：惩罚“真值为极性、却把概率压在 neu 上”。
60
61     实测误差结构：valid 上 299 个错误里 **167 个（56%）是把极性样本判成中性**
62     （预测为 neu 的 269 个样本里只有 102 个正确，neu precision 仅 0.36）。
63     该正则直接把 p_neu 从极性样本上挤走，把“中性<->极性”错误转化为“邻类极性”错误。
64     """
65     if not lam or lam <= 0:
66         return None
67     p = torch.softmax(logits, dim=-1)
68     m = y_cls != 1
69     if m.sum() < 1:
70         return None
71     return p[m, 1].mean()
72
73
74 # -----
75 # D4: 强度分级辅助头（CORAL 有序回归）
76 # 动机（实测）：61.7% 的样本  $|y| \leq 2/3$ ，其 ACC 仅 0.486； $|y|=1/3$  层更低到 0.361。
77 # 误差 84% 是邻类错 —— 模型分不清“完全中性”与“轻微极性”。把  $|y|$  按 1/3 步长
78 # 离散为 K 个等级做有序回归，等于直接监督“强度分辨率”，而不是让分类头自己猜。
79 # -----
80 def mag_level(y_reg, n_levels=10):
81     """ $|y| \rightarrow$  等级 k in  $[0, n\_levels-1]$  ( $|y|$  的离散档位，步长 1/3  $\rightarrow$   $n\_levels=10$ )。"""
82     k = torch.round(torch.abs(y_reg) * 3.0).long()
83     return k.clamp(0, int(n_levels) - 1)
84
85
86 def coral_loss(logits_mag, y_reg, n_levels=10):
87     """CORAL: 共享投影 + 有序偏置，对 K-1 个累计事件  $P(k > j)$  做二分类。
88
89     logits_mag: (B, n_levels-1)
90     """
91     k = mag_level(y_reg, n_levels)
92     levels = torch.arange(logits_mag.size(1), device=k.device).unsqueeze(0)
93     target = (k.unsqueeze(1) > levels).float()
94     return F.binary_cross_entropy_with_logits(logits_mag, target)
95
96
97 def mag_consistency(mu, logits_mag, n_levels=10):
98     """辅助一致性：累计概率得到的期望等级应与  $|\mu|$  对得上（弱监督，可选）。"""
99     p = torch.sigmoid(logits_mag) #  $P(k > j)$ 

```

```

100     exp_k = p.sum(dim=1)                                # E[k] = Sigma_j P(k>j)
101     return F.smooth_l1_loss(exp_k / 3.0, torch.abs(mu).detach(), beta=0.2)
102
103
104 def cls_D_loss(logits, y_cls, y_reg, cls_weight=None, label_smoothing=0.0,
105                 soft_tau=0.0, soft_mix=0.0, lam_emd=0.0, weak_alpha=0.0,
106                 lam_neusup=0.0):
107     """D 系列合成分类损失（针对弱标注样本 / 中性类）。
108
109     D2 soft_mix>0: 硬 CE 与强度软目标 KL 的凸组合
110     D2 lam_emd>0: 有序 EMD^2 项（内建序结构；实测 84% 错误只是邻类错）
111     D3 weak_alpha>0: 按  $w = 1 + \alpha \cdot (1 - |y|/3)$  给**弱标注样本**加权
112     D3' lam_neusup>0: 中性抑制正则（代价敏感，专治“极性样本被判成中性”）
113     """
114     hard = F.cross_entropy(logits, y_cls, weight=cls_weight,
115                             reduction="none", label_smoothing=label_smoothing)
116     q = None
117     if soft_mix and soft_mix > 0:
118         q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
119         soft = -(q * F.log_softmax(logits, dim=-1)).sum(-1)
120         per = (1.0 - float(soft_mix)) * hard + float(soft_mix) * soft
121     else:
122         per = hard
123     if weak_alpha and weak_alpha > 0:
124         w = 1.0 + float(weak_alpha) * (1.0 - (torch.abs(y_reg) / 3.0).clamp(0, 1))
125         per = per * w
126     loss = per.mean()
127     if lam_emd and lam_emd > 0:
128         if q is None:
129             q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
130             loss = loss + float(lam_emd) * emd2_loss(logits, q)
131     if lam_neusup and lam_neusup > 0:
132         ls = neu_suppress_loss(logits, y_cls, lam_neusup)
133         if ls is not None:
134             loss = loss + float(lam_neusup) * ls
135     return loss
136
137
138
139 def consistency_loss(mu, logits):
140     """极性-强度一致性。
141
142     实验事实：附件2的 sign(强度) 与三分类标签一致率为 **1.0000**（n=4850），
143     即“极性 == 强度符号”是硬约束。因此 tanh(mu) 与分类头的期望极性 E[cls]=P(pos)-P(neg)
144     应当一致；该正则把回归头与分类头对齐。
145     """

```

```

146     p = torch.softmax(logits, dim=-1)
147     e = p[:, 2] - p[:, 0] # in[-1,1]
148     return F.mse_loss(torch.tanh(mu), e)
149
150
151 def weighted_smooth_l1(mu, y, beta=0.5, alpha=0.0, gamma=0.5):
152     """按 |y| 加权的 SmoothL1: 把训练重心移向误差主体（强情感样本）。
153
154     alpha=0 时退化为普通 SmoothL1（与 MAE 目标对齐）。
155     """
156     loss = F.smooth_l1_loss(mu, y, beta=beta, reduction="none")
157     if alpha and alpha > 0:
158         w = 1.0 + float(alpha) * torch.abs(y).pow(float(gamma))
159         loss = loss * w
160     return loss.mean()
161
162
163 def masked_l1(pred, target, sel):
164     """sel:(B,L) bool, 只对选中位置求 L1。"""
165     if sel.sum() < 1:
166         return pred.new_zeros(())
167     p = pred[sel]
168     t = target[sel]
169     return (p - t).abs().mean()
170
171
172 def recon_loss(rec, x, miss, pad, use_missing=True):
173     """重建损失: use_missing=True 时只算缺失位置, 否则只算可见位置。"""
174     total, n = rec[list(rec.keys())[0]].new_zeros((), 0)
175     for m in MODALITIES:
176         sel = (miss[m] & ~pad[m]) if use_missing else (~miss[m] & ~pad[m])
177         if sel.sum() < 1:
178             continue
179         total = total + masked_l1(rec[m], x[m], sel)
180         n += 1
181     return total / max(n, 1)
182
183
184 def masked_mean(h, valid):
185     w = valid.float().unsqueeze(-1)
186     return (h * w).sum(dim=1) / w.sum(dim=1).clamp(min=1.0)
187
188
189 def sim_loss(share, pad):
190     """共享表示相似性: 把各模态“模态不变”部分拉到一起。"""
191     pooled = {m: masked_mean(share[m], ~pad[m]) for m in MODALITIES}

```

```

192     loss, n = pooled["text"].new_zeros(()), 0
193     keys = list(pooled.keys())
194     for i in range(len(keys)):
195         for j in range(i + 1, len(keys)):
196             loss = loss + F.mse_loss(pooled[keys[i]], pooled[keys[j]])
197             n += 1
198     return loss / max(n, 1)
199
200
201 def diff_loss(share, spec, pad):
202     """共享/特有正交：用余弦平方，避免尺度影响。"""
203     loss, n = None, 0
204     for m in MODALITIES:
205         a, b = share[m], spec[m]
206         cos = F.cosine_similarity(a, b, dim=-1)
207         w = (~pad[m]).float()
208         v = ((cos ** 2) * w).sum() / w.sum().clamp(min=1.0)
209         loss = v if loss is None else loss + v
210         n += 1
211     return loss / max(n, 1)
212
213
214 def balance_loss(gate):
215     """专家负载均衡（熵形式），防止专家坍塌。"""
216     gbar = gate.mean(dim=0)
217     E = gate.size(1)
218     return -(gbar * torch.log(gbar + 1e-8)).sum() * E
219
220
221 def total_loss(out, batch, cfg, lam_kd=0.0, teacher_out=None):
222     """组装总损失；返回 (loss, 明细 dict)。"""
223     mu, logvar, logits = out["mu"], out["logvar"], out["logits"]
224     x, miss, pad = batch["x"], batch["miss"], batch["pad"]
225
226     l_reg_main = weighted_smooth_l1(mu, batch["y_reg"], beta=getattr(cfg, "reg_beta", 0.5),
227                                     alpha=getattr(cfg, "y_weight_alpha", 0.0),
228                                     gamma=getattr(cfg, "y_weight_gamma", 0.5))
229     l_reg_var = reg_nll(mu, logvar, batch["y_reg"])
230     l_reg = l_reg_main + getattr(cfg, "lam_var", 0.1) * l_reg_var
231     cw = getattr(cfg, "cls_weight", None)
232     if cw is not None:
233         cw = torch.tensor(cw, dtype=logits.dtype, device=logits.device)
234     soft_mix = float(getattr(cfg, "soft_mix", 0.0) or 0.0)
235     lam_emd = float(getattr(cfg, "lam_emd", 0.0) or 0.0)
236     weak_alpha = float(getattr(cfg, "weak_alpha", 0.0) or 0.0)
237     lam_neusup = float(getattr(cfg, "lam_neusup", 0.0) or 0.0)

```

```

238 if soft_mix > 0 or lam_emd > 0 or weak_alpha > 0 or lam_neusup > 0:
239     l_cls = cls_D_loss(logits, batch["y_cls"], batch["y_reg"], cw,
240                        label_smoothing=getattr(cfg, "label_smoothing", 0.0),
241                        soft_tau=float(getattr(cfg, "soft_tau", 0.0) or 0.0),
242                        soft_mix=soft_mix, lam_emd=lam_emd,
243                        weak_alpha=weak_alpha, lam_neusup=lam_neusup)
244 else:
245     l_cls = cls_ce(logits, batch["y_cls"], cw, label_smoothing=getattr(cfg, "
label_smoothing", 0.0))
246 zero = l_reg.new_zeros(())
247 has = out.__contains__
248 # D4: 强度分级辅助头 (CORAL 有序回归)
249 lam_mag = float(getattr(cfg, "lam_mag", 0.0) or 0.0)
250 l_mag = (coral_loss(out["logits_mag"], batch["y_reg"],
251                    n_levels=int(getattr(cfg, "mag_levels", 10) or 10))
252           if (lam_mag > 0 and has("logits_mag")) else zero)
253 # 二分类极性辅助头: 直接监督 "是否负面", 与 Acc-2 的判定目标一致
254 lam_pol = float(getattr(cfg, "lam_pol", 0.0) or 0.0)
255 l_pol = zero
256 if lam_pol > 0 and has("pol"):
257     tneg = (batch["y_reg"] < 0).float()
258     pw = ((1.0 - tneg).sum() / tneg.sum().clamp(min=1.0)).detach()
259     l_pol = F.binary_cross_entropy_with_logits(out["pol"], tneg,
260                                                pos_weight=pw)
261 # 模块可缺省: A0/A1 朴素基线没有重建/分解/专家, 对应项自动跳过 (置 0)
262 l_rec = recon_loss(out["rec"], x, miss, pad, use_missing=True) if has("rec") else zero
263 l_rec0 = recon_loss(out["rec"], x, miss, pad, use_missing=False) if has("rec") else zero
264 l_sim = (sim_loss(out["share"], out["pad_ds"]))
265     if (has("share") and has("pad_ds")) else zero)
266 l_diff = (diff_loss(out["share"], out["spec"], out["pad_ds"]))
267     if (has("share") and has("spec") and has("pad_ds")) else zero)
268 l_bal = balance_loss(out["gate"]) if has("gate") else zero
269 lam_cons = float(getattr(cfg, "lam_cons", 0.0))
270 l_cons = consistency_loss(mu, logits) if lam_cons > 0 else zero
271
272 loss = (l_reg + cfg.lam_cls * l_cls + cfg.lam_rec * l_rec + cfg.lam_rec0 * l_rec0
273         + cfg.lam_sim * l_sim + cfg.lam_diff * l_diff + cfg.lam_bal * l_bal
274         + lam_cons * l_cons + lam_mag * l_mag + lam_pol * l_pol)
275
276 detail = dict(reg=float(l_reg.detach()), reg_main=float(l_reg_main.detach()),
277              reg_var=float(l_reg_var.detach()), cls=float(l_cls.detach()),
278              cons=float(l_cons.detach()), mag=float(l_mag.detach()),
279              pol=float(l_pol.detach()),
280              rec=float(l_rec.detach()),
281              rec0=float(l_rec0.detach()), sim=float(l_sim.detach()),
282              diff=float(l_diff.detach()), bal=float(l_bal.detach()))

```

```

283
284     if teacher_out is not None and lam_kd > 0:
285         tau = cfg.kd_tau
286         l_kd_logit = F.kl_div(F.log_softmax(logits / tau, dim=-1),
287                               F.softmax(teacher_out["logits"] / tau, dim=-1),
288                               reduction="batchmean") * tau * tau
289         l_kd_reg = F.mse_loss(mu, teacher_out["mu"])
290         # 关系蒸馏: 样本间相似度矩阵 (CMAD 相关性感知的简化实现)
291         st = F.normalize(out["z"], dim=-1)
292         tt = F.normalize(teacher_out["z"], dim=-1)
293         sim_s, sim_t = st @ st.t(), tt @ tt.t()
294         l_kd_rel = F.mse_loss(sim_s, sim_t)
295         l_kd = l_kd_logit + l_kd_reg + 0.1 * l_kd_rel      # 关系蒸馏降权, 避免量级压过任务损
失
296         loss = loss + lam_kd * l_kd
297         detail.update(kd=float(l_kd), kd_logit=float(l_kd_logit),
298                      kd_reg=float(l_kd_reg), kd_rel=float(l_kd_rel))
299     return loss, detail
300
301
302 # -----
303 # 指标
304 # -----
305 def _to_np(t):
306     return t.detach().cpu().numpy() if torch.is_tensor(t) else np.asarray(t)
307
308
309 def polar_from_score(score, theta):
310     """连续强度 -> 三分类 (题目定义: <0 负, =0 中, >0 正; theta 容差在 valid 上标定)。"""
311     out = np.ones_like(score, dtype=np.int64)
312     out[score < -theta] = 0
313     out[score > theta] = 2
314     return out
315
316
317 def tune_threshold(score, label_cls, grid=None):
318     """在验证集上按 Macro-F1 选择决策阈值。"""
319     from sklearn.metrics import f1_score
320     if grid is None:
321         grid = np.arange(0.0, 2.01, 0.05)
322     best, best_theta = -1.0, 0.0
323     for th in grid:
324         pred = polar_from_score(score, th)
325         f1 = f1_score(label_cls, pred, average="macro", zero_division=0)
326         if f1 > best:
327             best, best_theta = f1, float(th)

```

```

328     return best_theta, best
329
330
331 def compute_metrics(y_true_reg, y_pred_reg, y_true_cls, theta=0.0, y_pred_cls=None):
332     from sklearn.metrics import accuracy_score, f1_score
333     from scipy.stats import pearsonr
334     yt = _to_np(y_true_reg).astype(np.float64)
335     yp = _to_np(y_pred_reg).astype(np.float64)
336     tc = _to_np(y_true_cls).astype(np.int64)
337
338     mae = float(np.abs(yt - yp).mean())
339     try:
340         corr = float(pearsonr(yt, yp)[0])
341     except Exception:
342         corr = float("nan")
343     # CCC
344     mt, mp = yt.mean(), yp.mean()
345     vt, vp = yt.var(), yp.var()
346     cov = ((yt - mt) * (yp - mp)).mean()
347     ccc = float(2 * cov / (vt + vp + (mt - mp) ** 2 + 1e-12))
348
349     # 极性判别有两套口径:
350     # (A) theta 阈值口径: |y_hat| <= theta 判为 Neutral (theta 在 valid 上按 Macro-F1 选出)
351     # —— 与提交 CSV 的标签完全一致, 故作为**主指标**
352     # (B) 分类头 argmax 口径: 直接取分类头三类的最大值
353     pc_th = polar_from_score(yp, theta).astype(np.int64)
354     res = dict(mae=mae, pearson=corr, ccc=ccc,
355               acc=float(accuracy_score(tc, pc_th)),
356               f1=float(f1_score(tc, pc_th, average="macro", zero_division=0)),
357               theta=float(theta))
358     f1s = f1_score(tc, pc_th, average=None, zero_division=0, labels=[0, 1, 2])
359     res["f1_neg"], res["f1_neu"], res["f1_pos"] = [float(x) for x in f1s]
360     if y_pred_cls is not None:
361         pc_h = _to_np(y_pred_cls).astype(np.int64)
362         res["acc_head"] = float(accuracy_score(tc, pc_h))
363         res["f1_head"] = float(f1_score(tc, pc_h, average="macro", zero_division=0))
364     res["n"] = int(len(yt))
365     return res
366
367
368 def format_metrics(res, tag=""):
369     s = ("%s MAE=%.4f Pearson=%.4f CCC=%.4f | ACC(theta)=%.4f MacroF1(theta)=%.4f"
370         % (tag, res["mae"], res["pearson"], res["ccc"], res["acc"], res["f1"]))
371     if "acc_head" in res:
372         s += " | ACC(分类头)=%.4f MacroF1(分类头)=%.4f" % (res["acc_head"], res["f1_head"])
373     s += (" | F1 neg/neu/pos=%.3f/%.3f/%.3f theta=%.2f n=%d"

```

```

374         % (res["fl_neg"], res["fl_neu"], res["fl_pos"], res["theta"], res["n"]))
375     return s

```

C.3 问题二：训练与推理主流程（缺失课程学习、EMA、早停）

code/02_model/train.py（共 476 行）

```

1  # -*- coding: utf-8 -*-
2  """
3  问题2 训练主程序：教师（全模态）->学生（课程学习+缺失注入+蒸馏）->评价->缺失因素扫描
4
5  用法示例：
6  python train.py --data_dir E:\数学建模\data --version aligned
7  python train.py --version unaligned --epochs_teacher 30 --epochs_student 50 --sweep 1
8  python train.py --limit 64 --epochs_teacher 2 --epochs_student 2 --seed 42 # 冒烟测试
9  """
10 import json
11 import os
12 import sys
13 import time
14
15 import numpy as np
16 import torch
17 from torch.utils.data import DataLoader
18
19 sys.path.append(os.path.dirname(os.path.abspath(__file__)))
20 from config import get_config # noqa: E402
21 from data_utils import load_all, MoseiDataset, collate, describe # noqa: E402
22 from missing_sim import (inject_missing, batch_missing_ratios, # noqa: E402
23                          make_scenario_masks, crop_batch,
24                          inject_modality_dropout)
25 from model import MRFNet, PlainFusion # noqa: E402
26 from losses import (total_loss, compute_metrics, tune_threshold, # noqa: E402
27                    format_metrics)
28
29 MODALITIES = ("text", "audio", "vision")
30
31
32 # -----
33 def set_seed(seed):
34     np.random.seed(seed)
35     torch.manual_seed(seed)
36     torch.cuda.manual_seed_all(seed)
37
38
39 def make_loader(split, cfg, shuffle, augment=False):

```

```

40     ds = MoseiDataset(split, augment=augment)
41     return DataLoader(ds, batch_size=cfg.batch_size, shuffle=shuffle,
42                       num_workers=cfg.num_workers, collate_fn=collate,
43                       drop_last=False)
44
45
46 def prep(batch, device):
47     x = {m: batch[m].to(device) for m in MODALITIES}
48     miss = {m: batch["miss_" + m].to(device).bool() for m in MODALITIES}
49     pad = {m: batch["pad_" + m].to(device) for m in MODALITIES}
50     for m in MODALITIES:
51         pad[m] = ensure_valid(pad[m])          # 防全遮 NaN
52     return dict(x=x, miss=miss, pad=pad,
53               y_reg=batch["y_reg"].to(device),
54               y_cls=batch["y_cls"].to(device),
55               idx=batch["idx"].to(device))
56
57
58 def ensure_valid(pad):
59     """保证每条样本每个模态至少有一个非填充位（否则注意力 key 全被屏蔽 -> NaN）。"""
60     allpad = pad.all(dim=1, keepdim=True)
61     return torch.where(allpad, torch.zeros_like(pad), pad)
62
63
64 def mask_feed(b, cfg, unaware=False):
65     """模型看到的”缺失指示”。
66
67     unaware=True（推理期 A0 基线）或 cfg.ablate_mask_indicator=True（A2 消融，训练期也不给）
68     时，把指示位置零：特征仍在缺失位被置零，但模型不知道”那段是缺失的”。
69
70     """
71     if unaware or getattr(cfg, "ablate_mask_indicator", False):
72         return {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
73     return b["miss"]
74
75 def stack_masks(b):
76     M = torch.stack([b["miss"][m] for m in MODALITIES], dim=-1)
77     P = torch.stack([b["pad"][m] for m in MODALITIES], dim=-1)
78     return M, P
79
80
81 def unstack_masks(b, M):
82     """把缺失/填充掩码写回，并把不可用位置的特征置零（屏蔽进模型的有效值）。"""
83     for i, m in enumerate(MODALITIES):
84         b["miss"][m] = M[..., i]
85         bad = (b["miss"][m] | b["pad"][m]).float().unsqueeze(-1)

```

```

86         b["x"][m] = b["x"][m] * (1.0 - bad)
87
88
89 def curriculum(epoch, cfg):
90     """课程学习的缺失注入参数。"""
91     for e0, e1, r0, r1, mode in cfg.curric:
92         if e0 <= epoch <= e1:
93             return (r0, r1, mode)
94     return (0.2, 0.5, "mixed")
95
96
97 # -----
98 def run_epoch(model, loader, cfg, device, optimizer=None, teacher=None,
99               lam_kd=0.0, inject_cfg=None, rng=None, ema=None, ema_decay=0.995,
100               epoch=None):
101     train = optimizer is not None
102     model.train(train)
103     if teacher is not None:
104         teacher.eval()
105     if rng is None:
106         rng = np.random.default_rng()
107     tot, cnt, details = 0.0, 0, {}
108     for batch in loader:
109         b = prep(batch, device)
110         x_full = {m: b["x"][m].clone() for m in MODALITIES}
111         if train and getattr(cfg, "crop_aug", False):
112             b = crop_batch(b, rng, min_keep=cfg.crop_min_keep)
113         if train and getattr(cfg, "noise_sigma", 0.0) > 0:
114             sig = float(cfg.noise_sigma)
115             for m in MODALITIES:
116                 b["x"][m] = b["x"][m] + torch.randn_like(b["x"][m]) * sig
117         M, P = stack_masks(b)
118         if inject_cfg is not None:
119             mix = None
120             if getattr(cfg, "rho_mix", None):
121                 ep = 10 ** 9 if epoch is None else int(epoch)
122                 if ep >= int(getattr(cfg, "rho_mix_epoch", 1) or 1):
123                     mix = cfg.rho_mix
124             M2, _ = inject_missing(M, P, inject_cfg[0], inject_cfg[1],
125                                   inject_cfg[2], tuple(cfg.pos_modes), rng=rng,
126                                   n_intervals=tuple(getattr(cfg, "n_intervals", (1, 3))),
127                                   rho_mix=mix)
128             M = M2
129         if train and getattr(cfg, "mod_dropout", 0.0) > 0:
130             M = inject_modality_dropout(M, P, cfg.mod_dropout, rng=rng, n_mods=1)
131         unstack_masks(b, M)

```

```

132
133     with torch.set_grad_enabled(train):
134         ac = autocast_ctx(cfg)
135         if ac is None:
136             mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"],
return_aux=True)
137         else:
138             with ac:
139                 mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"],
return_aux=True)
140                 mu, logvar, logits, aux = _to_fp32(mu), _to_fp32(logvar), _to_fp32(logits
), _to_fp32(aux)
141                 out = dict(aux)
142                 out.update(mu=mu, logvar=logvar, logits=logits)
143
144                 t_out = None
145                 if teacher is not None and lam_kd > 0:
146                     with torch.no_grad():
147                         zero = {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
148                         if ac is None:
149                             t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"],
return_aux=True)
150                         else:
151                             with ac:
152                                 t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"],
return_aux=True)
153                                 t_mu, t_logits = t_mu.float(), t_logits.float()
154                                 t_out = dict(mu=t_mu, logits=t_logits, z=t_aux["z"].float())
155
156                 loss, det = total_loss(out, b, cfg, lam_kd=lam_kd, teacher_out=t_out)
157
158     if train:
159         optimizer.zero_grad()
160         loss.backward()
161         torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
162         optimizer.step()
163         if ema is not None:                                     # 权重指数移动平均
164             with torch.no_grad():
165                 sd = model.state_dict()
166                 for k in ema:
167                     v = sd[k]
168                     if v.dtype.is_floating_point:
169                         ema[k].mul_(ema_decay).add_(v.detach().float(), alpha=1.0 -
ema_decay)
170                     else:
171                         ema[k].copy_(v.detach())

```

```

172
173     tot += float(loss.detach())
174     cnt += 1
175     for k, v in det.items():
176         details[k] = details.get(k, 0.0) + v
177     return tot / max(cnt, 1), {k: v / max(cnt, 1) for k, v in details.items()}
178
179
180 @torch.no_grad()
181 def predict(model, loader, cfg, device, split=None, scenario=None, unaware=False):
182     model.eval()
183     ac = autocast_ctx(cfg)
184     S, CL, YR, YC, ID, RH = [], [], [], [], [], []
185     for batch in loader:
186         b = prep(batch, device)
187         M, P = stack_masks(b)
188         if scenario is not None:
189             mtype, pos, rho = scenario
190             M = make_scenario_masks(M, P, mtype, pos, rho, seed=1234)
191         unstack_masks(b, M)
192         feed = mask_feed(b, cfg, unaware=unaware)
193         if ac is None:
194             mu, _, logits = model(b["x"], feed, b["pad"])
195         else:
196             with ac:
197                 mu, _, logits = model(b["x"], feed, b["pad"])
198             mu, logits = mu.float(), logits.float()
199         S.append(mu.cpu().numpy())
200         CL.append(logits.cpu().numpy())
201         YR.append(b["y_reg"].cpu().numpy())
202         YC.append(b["y_cls"].cpu().numpy())
203         RH.append(batch_missing_ratios(M, P).cpu().numpy())
204         if split is not None:
205             idx = b["idx"].cpu().numpy()
206             ID.extend([split["ids"][i] for i in idx])
207     out = dict(score=np.concatenate(S), logits=np.concatenate(CL),
208               y_reg=np.concatenate(YR), y_cls=np.concatenate(YC),
209               rho=np.concatenate(RH))
210     out["ids"] = np.array(ID) if ID else None
211     return out
212
213
214 def report(pred, theta, tag=""):
215     res = compute_metrics(pred["y_reg"], pred["score"], pred["y_cls"],
216                           theta=theta, y_pred_cls=pred["logits"].argmax(-1))
217     print(format_metrics(res, tag=tag))

```

```

218     return res
219
220
221 # -----
222 def build_model(cfg, dims):
223     kind = getattr(cfg, "model_kind", "mrf")
224     if kind == "plain":
225         # A0/A1 基线：朴素拼接 + MLP；—ablate_mask 时连“缺失指示”这一输入通道也不给
226         return PlainFusion(dims=dims, hidden=cfg.hidden, dropout=cfg.dropout,
227                             target_len=cfg.target_len,
228                             use_indicator=not getattr(cfg, "ablate_mask_indicator", False)).to(
229                                 cfg.device)
230     return MRFNet(dims=dims, hidden=cfg.hidden, nhead=cfg.nhead,
231                   enc_layers=cfg.enc_layers, n_experts=cfg.n_experts,
232                   dropout=cfg.dropout, downs=cfg.down_factor_map,
233                   target_len=cfg.target_len,
234                   mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
235                   ms_pool=bool(getattr(cfg, "ms_pool", False)),
236                   ms_windows=tuple(getattr(cfg, "ms_windows", (5, 25, 50))),
237                   use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0).to(cfg.device)
238
239 def autocast_ctx(cfg):
240     """CPU bfloat16 autocast 上下文（本机 AMD Zen5 支持 AVX512-BF16）。
241
242     注意：CPU 上 PyTorch 的“半精度加速”是 **bfloat16** 而不是 float16
243     （AVX512-FP16 仅 Intel Sapphire Rapids 有；AMD 无 fp16 原生通路）。
244     bfloat16 与 fp32 动态范围相同（8 位指数），无需 GradScaler。
245     """
246     if not getattr(cfg, "bfloat16", False):
247         return None
248     try:
249         return torch.autocast(device_type="cpu", dtype=torch.bfloat16)
250     except Exception as e:
251         # pragma: no cover
252         print("[WARN] bfloat16 autocast 不可用： %s" % e)
253         return None
254
255 def _to_fp32(obj):
256     """把模型输出递归转回 float32，保证自定义损失在 fp32 下计算（可预测、无精度噪声）。"""
257     if torch.is_tensor(obj):
258         return obj.float() if obj.dtype.is_floating_point else obj
259     if isinstance(obj, dict):
260         return {k: _to_fp32(v) for k, v in obj.items()}
261     if isinstance(obj, (list, tuple)):
262         return type(obj)(_to_fp32(v) for v in obj)

```

```

263     return obj
264
265
266 def run_experiment(cfg, seed, data):
267     print("\n" + "=" * 78)
268     print("[SEED %d] version=%s device=%s hidden=%d epochs(t/s)=%d/%d distill=%s"
269           % (seed, cfg.version, cfg.device, cfg.hidden,
270              cfg.epochs_teacher, cfg.epochs_student, getattr(cfg, "use_distill", True)))
271     print("=" * 78)
272     set_seed(seed)
273     tr, va, te = data
274     if cfg.cls_balanced:
275         cnt = np.bincount(np.asarray(tr["labels_cls"]).astype(np.int64), minlength=3).astype
276         (np.float64)
277         w = cnt.sum() / (3.0 * np.maximum(cnt, 1.0))
278         w = np.clip(w, 0.2, 10.0) # 防止某类缺失时权重爆炸
279         cfg.cls_weight = [float(x) for x in w]
280         print("[CLS] 类别计数=%s 权重=%s" % (cnt.astype(int).tolist(),
281                                             [round(x, 3) for x in cfg.cls_weight]))
282     dims = {m: tr["D"][m] for m in MODALITIES}
283     print("dims=%s" % dims)
284     train_loader = make_loader(tr, cfg, shuffle=True, augment=True)
285     valid_loader = make_loader(va, cfg, shuffle=False)
286     test_loader = make_loader(te, cfg, shuffle=False)
287
288     tag = "s%d" % seed
289     hist = []
290
291     # ----- 1) 教师: 全模态 -----
292     teacher = build_model(cfg, dims)
293     opt = torch.optim.AdamW([
294         {"params": [p for n, p in teacher.named_parameters() if "head_" not in n], "lr": cfg.
295         lr_backbone},
296         {"params": [p for n, p in teacher.named_parameters() if "head_" in n], "lr": cfg.
297         lr_head}],
298         weight_decay=cfg.weight_decay)
299     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.
300     epochs_teacher))
301     best_mae, best_state, bad = 1e9, None, 0
302     for ep in range(1, cfg.epochs_teacher + 1):
303         t0 = time.time()
304         l, det = run_epoch(teacher, train_loader, cfg, cfg.device, optimizer=opt,
305         inject_cfg=None)
306         sched.step()
307         pv = predict(teacher, valid_loader, cfg, cfg.device, split=va)
308         mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())

```

```

304     print("[T ep%02d] loss=%.4f mae=%.4f rec=%.4f (%.1fs)"
305           % (ep, l, mae, det.get("rec", 0.0), time.time() - t0))
306     hist.append(dict(phase="teacher", epoch=ep, loss=l, valid_mae=mae, **det))
307     if mae < best_mae - 1e-4:
308         best_mae, bad = mae, 0
309         best_state = {k: v.detach().cpu().clone() for k, v in teacher.state_dict().
items()}
310     else:
311         bad += 1
312         if bad >= cfg.patience:
313             print("[T] early stop @ep%d" % ep)
314             break
315     if best_state is not None:
316         teacher.load_state_dict(best_state)
317     print("[T] best valid MAE = %.4f" % best_mae)
318
319     # ----- 2) 学生: 缺失鲁棒 (课程学习 + 蒸馏) -----
320     student = build_model(cfg, dims)
321     opt = torch.optim.AdamW([
322         {"params": [p for n, p in student.named_parameters() if "head_" not in n], "lr": cfg.
lr_backbone},
323         {"params": [p for n, p in student.named_parameters() if "head_" in n], "lr": cfg.
lr_head}],
324         weight_decay=cfg.weight_decay)
325     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.
epochs_student))
326     use_kd = getattr(cfg, "use_distill", True)
327     ema = None
328     if getattr(cfg, "ema", False):
329         ema = {k: v.detach().clone().float() for k, v in student.state_dict().items()}
330         print("[EMA] 启用权重滑动平均, decay=%.4f" % getattr(cfg, "ema_decay", 0.995))
331     best_mae, best_state, bad = 1e9, None, 0
332     for ep in range(1, cfg.epochs_student + 1):
333         t0 = time.time()
334         r0, r1, mode = curriculum(ep, cfg)
335         if getattr(cfg, "ablate_inject", False):          # A0: 干净数据训练, 不注入缺失
336             inject_cfg = None
337         else:
338             inject_cfg = (r0, r1, mode)
339         if use_kd and ep > cfg.kd_warmup:
340             lam_kd = cfg.lam_kd0 * float(np.exp(-(ep - 1 - cfg.kd_warmup) / cfg.
kd_decay_epochs))
341         else:
342             lam_kd = 0.0
343         l, det = run_epoch(student, train_loader, cfg, cfg.device, optimizer=opt,
344                           teacher=teacher, lam_kd=lam_kd, inject_cfg=inject_cfg,

```

```

345         rng=np.random.default_rng(seed * 1000 + ep),
346         ema=ema, ema_decay=float(getattr(cfg, "ema_decay", 0.995)),
347         epoch=ep)
348     sched.step()
349     if ema is not None:
350         _bk = {k: v.detach().clone() for k, v in student.state_dict().items()}
351         dt = next(student.parameters()).dtype
352         student.load_state_dict({k: v.to(dt) for k, v in ema.items()})
353         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
354         student.load_state_dict(_bk)
355     else:
356         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
357     mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())
358     print("[S ep%02d] loss=%0.4f mae=%0.4f rec=%0.4f kd=%0.4f | rho~[%0.2f,%0.2f] %s (%0.1fs)"
359           % (ep, l, mae, det.get("rec", 0.0), det.get("kd", 0.0), r0, r1, mode, time.
360             time() - t0))
361     hist.append(dict(phase="student", epoch=ep, loss=l, valid_mae=mae,
362                    rho_min=r0, rho_max=r1, lam_kd=lam_kd, **det))
363     if mae < best_mae - 1e-4:
364         best_mae, bad = mae, 0
365         best_state = {k: v.detach().cpu().clone() for k, v in student.state_dict().
366                       items()}
367     else:
368         bad += 1
369         if bad >= cfg.patience:
370             print("[S] early stop @ep%d" % ep)
371             break
372     if best_state is not None:
373         student.load_state_dict(best_state)
374     print("[S] best valid MAE = %0.4f%s" % (best_mae, " (EMA 权重)" if ema is not None else ""))
375
376     # ----- 3) 阈值选择 + 评价 -----
377     pv = predict(student, valid_loader, cfg, cfg.device, split=va)
378     theta, f1v = tune_threshold(pv["score"], pv["y_cls"])
379     print("[TH] theta=%0.2f(valid MacroF1=%0.4f)" % (theta, f1v))
380     res = {}
381     res["valid_clean"] = report(pv, theta, tag="[VALID clean]")
382     pt = predict(student, test_loader, cfg, cfg.device, split=te)
383     res["test_clean"] = report(pt, theta, tag="[TEST clean]")
384     res["theta"] = theta
385
386     # ----- 4) 缺失因素扫描（可选，用于第4节规律分析） -----
387     if cfg.sweep:
388         rows = []
389         pv0 = pv
390         for mtype in ["text", "audio", "vision", "text+audio", "text+vision", "audio+vision"]:

```

```

389         for pos in ["head", "middle", "tail", "random"]:
390             for rho in [0.1, 0.2, 0.3, 0.4, 0.5]:
391                 pa = predict(student, valid_loader, cfg, cfg.device, split=va,
392                             scenario=(mtype, pos, rho), unaware=False)
393                 pu = predict(student, valid_loader, cfg, cfg.device, split=va,
394                             scenario=(mtype, pos, rho), unaware=True)
395                 ra = report(pa, theta)
396                 ru = report(pu, theta)
397                 rows.append(dict(missing_type=mtype, position=pos, rho=rho,
398                                aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["
acc"],
399                                aware_corr=ra["pearson"],
400                                unaware_mae=ru["mae"], unaware_f1=ru["f1"],
401                                unaware_acc=ru["acc"], unaware_corr=ru["pearson"])))
402
403     import pandas as pd
404     df = pd.DataFrame(rows)
405     df.to_csv(os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag), index=False)
406     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag))
407
408     # 代表性场景在 test 上的"缺失泛化"验证 (附件2 test 有标签, 可量化)
409     trows = []
410     for mtype, pos, rho in [("text", "middle", 0.3), ("audio", "random", 0.3),
411                             ("vision", "random", 0.3), ("text+audio", "random", 0.3),
412                             ("audio+vision", "middle", 0.3), ("text+audio+vision", "random",
0.3)]:
413         pa = predict(student, test_loader, cfg, cfg.device, split=te,
414                     scenario=(mtype, pos, rho), unaware=False)
415         pu = predict(student, test_loader, cfg, cfg.device, split=te,
416                     scenario=(mtype, pos, rho), unaware=True)
417         ra, ru = report(pa, theta), report(pu, theta)
418         trows.append(dict(missing_type=mtype, position=pos, rho=rho,
419                          aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["acc"],
420                          unaware_mae=ru["mae"], unaware_f1=ru["f1"], unaware_acc=ru["
acc"])))
421     pd.DataFrame(trows).to_csv(os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag),
index=False)
422     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag))
423     res["sweep"] = rows
424
425     # ----- 5) 保存 -----
426     ckpt = os.path.join(cfg.out_dir, "student_%s.pt" % tag)
427     state = student.state_dict()
428     if getattr(cfg, "save_half", False):
429         state = {k: (v.half() if v.is_floating_point() else v) for k, v in state.items()}
430     torch.save(dict(state=state, dims=dims, theta=theta,
stats=tr["stats"], cfg=cfg.__dict__, model_kind=getattr(cfg, "model_kind"

```

```

, "mrf"),
431         mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
432         ms_pool=bool(getattr(cfg, "ms_pool", False)),
433         ms_windows=list(getattr(cfg, "ms_windows", (5, 25, 50))),
434         use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0,
435         version=cfg.version, seed=seed, half=bool(getattr(cfg, "save_half",
False))), ckpt)
436 print("[SAVE] %s (%.2fMB)" % (ckpt, os.path.getsize(ckpt) / 1e6))
437 import pandas as pd
438 pd.DataFrame(hist).to_csv(os.path.join(cfg.out_dir, "history_%s.csv" % tag), index=False)
439 with open(os.path.join(cfg.out_dir, "metrics_%s.json" % tag), "w", encoding="utf-8") as f:
440     json.dump({k: v for k, v in res.items() if k != "sweep"}, f,
441               ensure_ascii=False, indent=2)
442 return res
443
444
445 def main():
446     cfg = get_config()
447     torch.set_num_threads(max(1, os.cpu_count() or 4))
448     print("[CFG] %s" % json.dumps({k: v for k, v in cfg.__dict__.items()
449                                   if isinstance(v, (int, float, str, bool))},
450                                   ensure_ascii=False, indent=2))
451     print("[DATA] 读取 %s" % cfg.pkl_path)
452     tr, va, te = load_all(cfg)
453     for name, s in (("train", tr), ("valid", va), ("test", te)):
454         print(describe(s, name))
455
456     all_res = []
457     for seed in cfg.seeds:
458         all_res.append(run_experiment(cfg, seed, (tr, va, te)))
459
460     if len(all_res) > 1:
461         print("\n" + "=" * 78)
462         print("[SUMMARY] %d seeds: mean +/- std" % len(all_res))
463         keys = ["mae", "pearson", "ccc", "acc", "fl"]
464         for split in ("valid_clean", "test_clean"):
465             line = split + ":"
466             for k in keys:
467                 v = np.array([r[split][k] for r in all_res])
468                 line += "%s=%.4f+/-%.4f " % (k, v.mean(), v.std())
469             print(line)
470         with open(os.path.join(cfg.out_dir, "summary.json"), "w", encoding="utf-8") as f:
471             json.dump([{"k: v for k, v in r.items() if k != "sweep"} for r in all_res],
472                       f, ensure_ascii=False, indent=2)
473
474

```

```

475 if __name__ == "__main__":
476     main()

```

C.4 问题三：骨干加载与批构造（三源解释的公共入口）

code/03_xai/xai_common.py（共 247 行）

```

1  # -*- coding: utf-8 -*-
2  r"""问题3 共享工具：骨干加载、split 构建、掩码/遮挡算子、目标标量、缓存读写。
3
4  设计要点（与 02_model 的推理链路严格一致，保证”解释的是同一个模型”）：
5  * 复用 ‘runtime.load_ensemble’ 与 ‘train.prep/stack_masks/unstack_masks’，
6    不重写前向路径，避免”解释的模型 != 预测的模型”；
7  * 屏蔽模式统一用 ‘miss=1’（模型在训练时见过的”整模式缺失”语义），
8    并同步把特征置零（与 ‘unstack_masks’ 的做法一致）；
9  * 目标标量支持 ‘mu’（强度，主口径）与 ‘p_pos/p_neg/pol/p_max’（极性，对照口径）。
10 """
11 import glob
12 import json
13 import os
14 import sys
15
16 import numpy as np
17 import torch
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 ROOT = os.path.dirname(CODE)
22 for p in (CODE, os.path.join(CODE, "02_model")):
23     if p not in sys.path:
24         sys.path.append(p)
25
26 from data_utils import load_any, MoseiDataset, collate # noqa: E402
27 from runtime import load_ensemble, predict_split # noqa: E402
28 from train import prep, stack_masks, unstack_masks # noqa: E402
29
30 MODALITIES = ("text", "audio", "vision")
31 LABELS = ("Negative", "Neutral", "Positive")
32 TARGETS = ("mu", "p_pos", "p_neg", "p_pol", "p_max")
33
34
35 # -----
36 # 骨干与数据
37 # -----
38 def load_backbone(ckpt_dir=None, device="cpu", verbose=True):
39     ckpt_dir = ckpt_dir or os.path.join(ROOT, "runs", "ens_top2")

```

```

40 ckpts = sorted(glob.glob(os.path.join(ckpt_dir, "student_*.pt")))
41 if not ckpts:
42     raise SystemExit("未找到权重: %s/student_*.pt" % ckpt_dir)
43 models = load_ensemble(ckpts, device=device, verbose=verbose)
44 for m, _ in models:                                # 只解释输入，不需要参数梯度
45     for p in m.parameters():
46         p.requires_grad_(False)
47     theta = float(np.mean([ck["theta"] for _, ck in models]))
48     return models, models[0][1]["stats"], theta
49
50
51 def load_split(pk1, stats, key=None):
52     """读任意 pk1（附件2/3/4）并用 ckpt 内统计量归一化。"""
53     splits = load_any(pk1, norm_stats=stats)
54     if key is None:
55         key = "test" if "test" in splits else list(splits.keys())[0]
56     return splits[key], key
57
58
59 def make_batch(split, idxs, device="cpu", mask=True):
60     """按 DataLoader 同路径构造 batch（含 pad 有效位兜底）。
61
62     mask=True 时额外执行 'apply_masks'（把 miss|pad 位置的特征置零），
63     与 'infer_att3.py' 的推理链路**完全一致**——这是“解释的模型=预测的模型”的前提：
64     若不置零，填充位的 BERT 垃圾值会经编码器内的局部卷积渗入最后一个有效位，
65     造成解释与提交预测的口径偏差（实测最大分数差 0.049、1/20 标签不同）。
66     """
67     ds = MoseiDataset(split, augment=False)
68     b = prep(collate([ds[int(i)] for i in idxs]), device)
69     return apply_masks(b) if mask else b
70
71
72 def apply_masks(b):
73     """与推理链路一致：把 miss|pad 位置的特征置零。"""
74     M, _ = stack_masks(b)
75     unstack_masks(b, M)
76     return b
77
78
79 def valid_len(b, m, j):
80     """第 j 个样本、模态 m 的有效长度（非填充位）。"""
81     return int((~b["pad"][m][j]).sum().item())
82
83
84 # -----
85 # 掩码/遮挡算子

```

```

86 # -----
87 def mask_modalities(b, mods, mode="miss"):
88     """屏蔽若干模态。mode='miss' 置缺失指示；mode='pad' 整模态不可见。"""
89     out = dict(x={}, miss={}, pad={})
90     for k in ("y_reg", "y_cls", "idx"):
91         if k in b:
92             out[k] = b[k]
93     for m in MODALITIES:
94         out["x"][m] = b["x"][m].clone()
95         out["miss"][m] = b["miss"][m].clone()
96         out["pad"][m] = b["pad"][m].clone()
97     for m in mods:
98         if mode == "miss":
99             out["miss"][m] = torch.ones_like(b["miss"][m])
100         else:
101             p = torch.ones_like(b["pad"][m])
102             p[:, 0] = False # 安全位：避免注意力全遮 -> NaN
103             out["pad"][m] = p
104     for m in MODALITIES:
105         bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
106         out["x"][m] = out["x"][m] * (1.0 - bad)
107     return out
108
109
110 def occlude_window(b, m, start, width):
111     """把模态 m 的 [start, start+width) 段用相邻段替换（保持序列长度与形态）。
112
113     首段向右取邻段、末段向左取邻段；缺失/填充位保持不可用。
114     """
115     out = dict(x={}, miss={}, pad={})
116     for k in ("y_reg", "y_cls", "idx"):
117         if k in b:
118             out[k] = b[k]
119     L = b["x"][m].size(1)
120     for mm in MODALITIES:
121         out["x"][mm] = b["x"][mm].clone()
122         out["miss"][mm] = b["miss"][mm].clone()
123         out["pad"][mm] = b["pad"][mm].clone()
124     s = max(0, int(start))
125     e = min(L, s + int(width))
126     if e <= s:
127         return out
128     off = e - s
129     if e + off <= L:
130         src = slice(e, e + off)
131         dst = slice(s, e)

```

```

132 elif s - off >= 0:
133     src = slice(s - off, s)
134     dst = slice(s, e)
135 else:                                     # 序列过短: 退回均值替换
136     src, dst = None, slice(s, e)
137 for j in range(out["x"][m].size(0)):
138     if src is None:
139         avail = ~(b["miss"][m][j] | b["pad"][m][j])
140         fill = b["x"][m][j][avail].mean(dim=0) if avail.any() else torch.zeros_like(
141             b["x"][m][j][0])
142         out["x"][m][j, dst] = fill
143     else:
144         out["x"][m][j, dst] = b["x"][m][j][src]
145         out["miss"][m][j, dst] = False      # 被替换段视为"有信息"
146 bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
147 out["x"][m] = out["x"][m] * (1.0 - bad)
148 return out
149
150
151 # -----
152 # 前向与目标标量
153 # -----
154 def predict_batch(models, b, target="mu", aux_keys=()):
155     """集成前向。返回 score/prob/target (均为可微张量) 与可选 aux。"""
156     mus, ps = [], []
157     aux_sum = {k: None for k in aux_keys}
158     for m, _ in models:
159         if aux_keys:
160             mu, _, lg, ax = m(b["x"], b["miss"], b["pad"], return_aux=True)
161             for k in aux_keys:
162                 v = ax.get(k)
163                 if torch.is_tensor(v) and v.size(0) == mu.size(0):
164                     v = v.detach()
165                     aux_sum[k] = v if aux_sum[k] is None else aux_sum[k] + v
166         else:
167             mu, _, lg = m(b["x"], b["miss"], b["pad"])
168             mus.append(mu)
169             ps.append(torch.softmax(lg, dim=-1))
170     score = torch.stack(mus).mean(0)
171     prob = torch.stack(ps).mean(0)
172     if target == "mu":
173         t = score
174     elif target == "p_pos":
175         t = prob[:, 2]
176     elif target == "p_neg":
177         t = prob[:, 0]

```

```

178 elif target == "p_pol":
179     t = prob[:, 2] - prob[:, 0]
180 elif target == "p_max":
181     t = prob.max(dim=-1).values
182 else:
183     raise ValueError("未知 target=%s (可选 %s)" % (target, TARGETS))
184 out = dict(score=score, prob=prob, target=t)
185 for k in aux_keys:
186     if aux_sum[k] is not None:
187         out[k] = aux_sum[k] / len(models)
188 return out
189
190
191 # -----
192 # 重要性后处理与缓存
193 # -----
194 def norm_valid(w, n_valid):
195     """把重要性向量限制在有效位内并归一到 [0,1] (全零时返回全零)。"""
196     w = np.asarray(w, dtype=np.float64).copy()
197     n = int(max(1, min(len(w), n_valid)))
198     seg = w[:n]
199     w[n:] = 0.0
200     lo, hi = float(seg.min()), float(seg.max())
201     if hi - lo < 1e-12:
202         return np.zeros_like(w)
203     w[:n] = (seg - lo) / (hi - lo)
204     return w
205
206
207 def softmax1d(x, tau=1.0):
208     x = np.asarray(x, dtype=np.float64) / max(float(tau), 1e-9)
209     x = x - x.max()
210     e = np.exp(x)
211     return e / e.sum()
212
213
214 def save_npz(path, **arrays):
215     os.makedirs(os.path.dirname(path), exist_ok=True)
216     np.savez_compressed(path, **arrays)
217     return path
218
219
220 def load_npz(path):
221     with np.load(path) as z:
222         return {k: z[k] for k in z.files}
223

```

```

224
225 def save_json(path, obj):
226     os.makedirs(os.path.dirname(path), exist_ok=True)
227     with open(path, "w", encoding="utf-8") as f:
228         json.dump(obj, f, ensure_ascii=False, indent=1, default=_jsonable)
229     return path
230
231
232 def _jsonable(o):
233     if isinstance(o, (np.integer,)):
234         return int(o)
235     if isinstance(o, (np.floating,)):
236         return float(o)
237     if isinstance(o, np.ndarray):
238         return o.tolist()
239     return str(o)
240
241
242 def q3_meta(ckpt_dir, models, theta, extra=None):
243     """记录本次解释使用的骨干与阈值，供复现核对。"""
244     files = sorted(os.path.basename(p) for p in
245                    glob.glob(os.path.join(ckpt_dir, "student_*.pt")))
246     return dict(model_files=files, n_ckpt=len(models), ckpt_dir=str(ckpt_dir),
247                theta=round(float(theta), 4), extra=extra or {})

```

C.5 问题三：精确 Shapley 值（八子集全枚举）

code/03_xai/shapley.py（共 258 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S1: 模态级精确 Shapley 值（3 模态 -> 8 个子集全枚举）。
3
4  数学定义（论文 section5.3 用）：
5      v(S) 为只用模态子集 S 时模型的目标标量；模态 m 的 Shapley 值
6       $\phi_m = \sum_{S \subset M \setminus \{m\}} \frac{|S|!(n-|S|-1)!}{n!} \cdot [v(S \cup \{m\}) - v(S)]$ ,  $n=3$ 
7      精确性检验： $\sum_m \phi_m = v(M) - v(\{\})$ 。
8
9  本脚本一次算完 4 个口径组合（保证可比、避免重复前向）：
10     target in {mu(强度,主), p_pol = p_pos - p_neg(极性,对照)}
11     semantic in {miss(置缺失指示,主), pad(整模态不可见,对照)}
12  并输出两套归一化（ReLU 归一 / softmax|phi|）供后续按验证集“模态忠实性”择优。
13
14  两类效用：
15     * 样本级：v(S) 取该样本自身的预测 -> 逐样本 phi（写入附件4 解释 CSV）；
16     * 数据集级：v(S) 取验证集上的 -MAE 与 Macro-F1 -> 全局模态重要性（论文结论）。
17

```

```

18 用法:
19 python shapley.py --pkl <pkl> --name att4 --out_dir <runs/q3>
20 python shapley.py --pkl <附件2 aligned_50.pkl> --split valid --name valid --with_global
21 """
22 import argparse
23 import itertools
24 import math
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30
31 HERE = os.path.dirname(os.path.abspath(__file__))
32 if HERE not in sys.path:
33     sys.path.append(HERE)
34 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
35                          predict_batch, mask_modalities, norm_valid, softmax1d,
36                          save_npz, save_json, q3_meta, ROOT)
37
38 SUBSETS = [tuple(m for i, m in enumerate(MODALITIES) if (s >> i) & 1) for s in range(8)]
39 MASKS = list(range(8))
40 NAMES = {0: "none", 1: "text", 2: "audio", 4: "vision",
41          3: "text+audio", 5: "text+vision", 6: "audio+vision", 7: "all"}
42 WEIGHTS = {}
43
44
45 def mods_of(mask):
46     """位掩码->模态名列表 (text=1, audio=2, vision=4)。"""
47     return [MODALITIES[i] for i in range(3) if (mask >> i) & 1]
48
49
50 def shapley_weights(n=3):
51     """精确 Shapley 的组合权重  $w(S) = |S|!(n-|S|-1)!/n!$  (按位掩码索引)。"""
52     if n not in WEIGHTS:
53         w = {}
54         for i in range(3):
55             others = [j for j in range(3) if j != i]
56             tab = {}
57             for r in range(len(others) + 1):
58                 for S in itertools.combinations(others, r):
59                     tab[sum(1 << j for j in S)] = (math.factorial(len(S)) *
60                                                     math.factorial(n - len(S) - 1) /
61                                                     math.factorial(n))
62             w[i] = tab
63         WEIGHTS[n] = w

```

```

64     return WEIGHTS[n]
65
66
67 def phi_from_values(v):
68     """由 8 个子集的 v 值（按 mask 索引）算精确 Shapley 向量 (3,)。"""
69     W = shapley_weights(3)
70     phi = np.zeros(3)
71     for i in range(3):
72         for T, wt in W[i].items():
73             phi[i] += wt * (v[T | (1 << i)] - v[T])
74     return phi
75
76
77 def normalizers(phi):
78     """两套归一化（对 (N,3) 或 (3,) 的 phi 逐样本处理）。"""
79     a = np.asarray(phi, dtype=np.float64)
80     single = a.ndim == 1
81     a = a[None] if single else a
82     relu = np.maximum(a, 0.0)
83     s = relu.sum(1, keepdims=True)
84     pi_relu = np.where(s > 1e-12, relu / np.maximum(s, 1e-12), np.nan)
85     pi_soft = np.exp(np.abs(a) / 1.0)
86     pi_soft = pi_soft / pi_soft.sum(1, keepdims=True)
87     return (pi_relu[0], pi_soft[0]) if single else (pi_relu, pi_soft)
88
89
90 def run_batch(models, b, target, semantic):
91     """对一个 batch 算 8 个子集的 v 值 -> V (B,8)，并返回全模态输出。"""
92     B = b["x"]["text"].size(0)
93     V = np.zeros((B, 8))
94     full = None
95     for mask in MASKS:
96         bb = b if mask == 7 else mask_modalities(
97             b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
98         out = predict_batch(models, bb, target=target)
99         V[:, mask] = out["target"].detach().cpu().numpy()
100         if mask == 7:
101             full = dict(score=out["score"].detach().cpu().numpy(),
102                         prob=out["prob"].detach().cpu().numpy())
103     return V, full
104
105
106 def global_values(models, split, target, semantic, batch=64, device="cpu"):
107     """数据集级效用：v(S) = -MAE（回归）与 Macro-F1（分类头概率）。"""
108     from sklearn.metrics import f1_score
109     N = split["N"]

```

```

110 y = np.asarray(split["labels_reg"])
111 yc = np.asarray(split["labels_cls"])
112 v_mae, v_f1 = np.zeros(8), np.zeros(8)
113 for mask in MASKS:
114     preds, probs = [], []
115     for i0 in range(0, N, batch):
116         idxs = list(range(i0, min(i0 + batch, N)))
117         b = make_batch(split, idxs, device)
118         if mask != 7:
119             b = mask_modalities(
120                 b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
121         out = predict_batch(models, b, target=target)
122         preds.append(out["score"].detach().cpu().numpy())
123         probs.append(out["prob"].detach().cpu().numpy())
124     sc, pb = np.concatenate(preds), np.concatenate(probs)
125     v_mae[mask] = -float(np.abs(sc - y).mean())
126     try:
127         v_f1[mask] = float(f1_score(yc, pb.argmax(1), average="macro"))
128     except Exception:
129         v_f1[mask] = np.nan
130     print("    [GLOBAL] S=%-12s MAE=%.4f MacroF1=%.4f"
131           % (NAMES[mask], -v_mae[mask], v_f1[mask]))
132 return v_mae, v_f1
133
134
135 def main():
136     ap = argparse.ArgumentParser()
137     ap.add_argument("--pkl", default=None, help="默认附件2 aligned (用 --split valid)")
138     ap.add_argument("--split", default=None, help="pkl 内的 split 名 (附件2 需指定 valid)")
139     ap.add_argument("--name", default="att4", help="输出文件名后缀")
140     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
141     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
142     ap.add_argument("--batch_size", type=int, default=32)
143     ap.add_argument("--limit", type=int, default=0, help=">0 时只算前 N 条")
144     ap.add_argument("--device", default="cpu")
145     ap.add_argument("--with_global", action="store_true", help="额外算数据集级 Shapley")
146     ap.add_argument("--global_target", default="mu", choices=["mu", "p_pol"])
147     a = ap.parse_args()
148
149     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
150     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
151     split, key = load_split(pkl, stats, a.split)
152     if a.limit:
153         from data_utils import slice_split
154         split = slice_split(split, a.limit)
155     N = split["N"]

```

```

156 print("[DATA] %s/%s N=%d" % (os.path.basename(pk1), key, N))
157
158 combos = [("mu", "miss"), ("mu", "pad"), ("p_pol", "miss"), ("p_pol", "pad")]
159 store, v_store = {}, {}
160 score_all, prob_all = None, None
161 for target, semantic in combos:
162     tag = "%s_%s" % (target, semantic)
163     V = np.zeros((N, 8))
164     phis = np.zeros((N, 3))
165     sc, pb = [], []
166     for i0 in range(0, N, a.batch_size):
167         idxs = list(range(i0, min(i0 + a.batch_size, N)))
168         b = make_batch(split, idxs, a.device)
169         v, full = run_batch(models, b, target, semantic)
170         for r in range(len(idxs)):
171             phis[i0 + r] = phi_from_values(v[r])
172         V[i0:i0 + len(idxs)] = v
173         sc.append(full["score"])
174         pb.append(full["prob"])
175     store[tag], v_store[tag] = phis, V
176     resid = float(np.abs(phis.sum(1) - (V[:, 7] - V[:, 0])).max())
177     print(" [%-10s] phi 均值 文/语/视 = %+0.4f / %+0.4f / %+0.4f; 精确性残差 max=%0.2e"
178           % (tag, *phis.mean(0), resid))
179     assert resid < 1e-6, "Shapley 精确性检验失败"
180     if tag == "mu_miss":
181         score_all, prob_all = np.concatenate(sc), np.concatenate(pb)
182
183 # 主口径: mu + miss; 同时给出两套归一化
184 phi_main = store["mu_miss"]
185 pi_relu, pi_soft = normalizers(phi_main)
186 n_relu = int(np.isnan(pi_relu).any(1).sum())
187 print(" [NORM] ReLU 归一在全负样本上退化 %d/%d 条; softmax 口径无退化" % (n_relu, N))
188 miss_rate = np.zeros((N, 3))
189 for i in range(N):
190     for k, m in enumerate(MODALITIES):
191         span = float((~split["pad"][m][i]).sum())
192         if span <= 1.0: # 整条全零: 整模态缺失 (口径 D1)
193             miss_rate[i, k] = 1.0
194         else:
195             miss_rate[i, k] = float((split["miss"][m][i] & ~split["pad"][m][i]).sum())
196 / span
197
198 out = dict(ids=np.asarray(split["ids"]).astype(str),
199           score=score_all, prob=prob_all, phi_mu_miss=phi_main,
200           pi_relu=np.nan_to_num(pi_relu, nan=0.0), pi_soft=pi_soft,
201           miss_rate=miss_rate, subset_names=np.array([NAMES[m] for m in MASKS]))

```

```

201 for tag in store:
202     out["phi_" + tag] = store[tag]
203     out["v_" + tag] = v_store[tag]
204 path = save_npz(os.path.join(a.out_dir, "q3_shapley_%s.npz" % a.name), **out)
205
206 # CSV (逐样本)
207 import pandas as pd
208 df = pd.DataFrame({
209     "id": out["ids"],
210     "score": np.round(score_all, 4),
211     "prob_neg": np.round(prob_all[:, 0], 4), "prob_neu": np.round(prob_all[:, 1], 4),
212     "prob_pos": np.round(prob_all[:, 2], 4),
213     "main_modality": [MODALITIES[int(np.argmax(r))] for r in pi_soft],
214     "main_modality_relu": [MODALITIES[int(np.argmax(r))] if not np.isnan(r).any()
215                             else "" for r in pi_relu],
216     "pi_text": np.round(pi_soft[:, 0], 4), "pi_audio": np.round(pi_soft[:, 1], 4),
217     "pi_vision": np.round(pi_soft[:, 2], 4),
218     "phi_text": np.round(phi_main[:, 0], 4), "phi_audio": np.round(phi_main[:, 1], 4),
219     "phi_vision": np.round(phi_main[:, 2], 4),
220     "phi_text_padsem": np.round(store["mu_pad"][:, 0], 4),
221     "phi_audio_padsem": np.round(store["mu_pad"][:, 1], 4),
222     "phi_vision_padsem": np.round(store["mu_pad"][:, 2], 4),
223     "miss_text": np.round(miss_rate[:, 0], 4), "miss_audio": np.round(miss_rate[:, 1], 4),
224     "miss_vision": np.round(miss_rate[:, 2], 4),
225 })
226 csv = os.path.join(a.out_dir, "q3_shapley_%s.csv" % a.name)
227 os.makedirs(a.out_dir, exist_ok=True)
228 df.to_csv(csv, index=False, encoding="utf-8-sig")
229
230 report = dict(meta=q3_meta(a.ckpt_dir, models, theta,
231                             dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N)),
232               data=os.path.relpath(path, ROOT),
233               phi_mean={k: np.round(store[k].mean(0), 4).tolist() for k in store},
234               pi_soft_mean=np.round(pi_soft.mean(0), 4).tolist(),
235               pi_relu_mean=np.round(np.nan_to_num(pi_relu, nan=0.0).mean(0), 4).
236               tolist(),
237               pi_relu_degenerate=int(n_relu),
238               v_mean_mu_miss={NAMES[m]: round(float(v_store["mu_miss"][:, m].mean()),
239               4)
240                               for m in MASKS})
241
242 if a.with_global:
243     print("[GLOBAL] 数据集级效用 (%s / miss 语义) : " % a.global_target)
244     v_mae, v_f1 = global_values(models, split, a.global_target, "miss", a.batch_size,
245                                 a.device)
246     report["global"] = dict(target=a.global_target,
247                             v_negmae={NAMES[m]: round(float(v_mae[m]), 4) for m in

```

```

MASKS},
245         v_macrof1={NAMES[m]: round(float(v_f1[m]), 4) for m in
MASKS},
246         phi_negmae=np.round(phi_from_values(v_mae), 4).tolist(),
247         phi_macrof1=np.round(
248             phi_from_values(np.nan_to_num(v_f1)), 4).tolist())
249     print("    phi(-MAE) = %s" % report["global"]["phi_negmae"])
250     print("    phi(MacroF1) = %s" % report["global"]["phi_macrof1"])
251     save_json(os.path.join(a.out_dir, "q3_shapley_%s_report.json" % a.name), report)
252     print("[SAVE] %s\n[SAVE] %s" % (os.path.relpath(path, ROOT), os.path.relpath(csv, ROOT)
    ))
253     print("SHAPLEY_OK")
254     return 0
255
256
257 if __name__ == "__main__":
258     sys.exit(main())

```

C.6 问题三：积分梯度与片段级重要性

code/03_xai/ig_gradcam.py (共 134 行)

```

1  #-*- coding: utf-8 -*-
2  r"""S2: 片段级时间重要性的梯度类来源（注意力 / 积分梯度 / 注意力 x 梯度）。
3
4  三个来源（论文 section5.3 的 (a)(b) 两族）：
5  (a) 注意力源      w^att_k = alpha_k          （模型内生，仅作基线）
6  (a') 注意力 x 梯度源  w^agrad_k = alpha_k . ||d t/d x_k||_2    （Grad-CAM 式加权）
7  (b) 积分梯度      w^ig_k = ||(x_k - x'_k) (*) (1/L)Sigma_l grad_{x_k} t(x' + l/L(x-x'))||_2
8
9  基线 x': 'zero'（归一化空间中即训练集均值，主口径）/ 'mean'（本 batch 有效位均值，对照）。
10
11  用法：
12  python ig_gradcam.py --pk1 <pk1> --name att4 --steps 20 --target mu --out_dir <runs/q3>
13  """
14  import argparse
15  import os
16  import sys
17
18  import numpy as np
19  import torch
20
21  HERE = os.path.dirname(os.path.abspath(__file__))
22  if HERE not in sys.path:
23      sys.path.append(HERE)
24  from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa:

```

```

25         predict_batch, save_npz, save_json, q3_meta, ROOT)
26
27
28 def ig_batch(models, b, steps=20, baseline="zero", target="mu", need_agrad=True):
29     """返回 (w_ig (B,3,L), w_att (B,3,L), w_agrad (B,3,L))。"""
30     x0 = {m: b["x"][m].detach().clone() for m in MODALITIES}
31     if baseline == "zero":
32         base = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
33     else:
34         # mean: 该样本有效位均值
35         base = {}
36         for m in MODALITIES:
37             valid = (~(b["miss"][m] | b["pad"][m])).float().unsqueeze(-1)
38             mu = (x0[m] * valid).sum(1, keepdim=True) / valid.sum(1, keepdim=True).clamp(
39                 min=1)
40             base[m] = mu.expand_as(x0[m]).contiguous()
41         gsum = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
42         for l in range(1, steps + 1):
43             a = 1 / float(steps)
44             xs = {m: (base[m] + a * (x0[m] - base[m])).requires_grad_(True) for m in
45                 MODALITIES}
46             out = predict_batch(models, dict(x=xs, miss=b["miss"], pad=b["pad"]), target=target
47                 )
48             g = torch.autograd.grad(out["target"].sum(), [xs[m] for m in MODALITIES],
49                 retain_graph=False)
50             for i, m in enumerate(MODALITIES):
51                 gsum[m] = gsum[m] + g[i].detach()
52             w_ig = torch.stack([(x0[m] - base[m]) * (gsum[m] / steps)].norm(dim=-1)
53                 for m in MODALITIES], dim=1) # (B,3,L)
54
55             # 注意力源 + 注意力 x 梯度源 (一次前向 + 一次反向)
56             xg = {m: x0[m].clone().requires_grad_(True) for m in MODALITIES}
57             out = predict_batch(models, dict(x=xg, miss=b["miss"], pad=b["pad"]),
58                 target=target, aux_keys=("alpha",))
59             grads = torch.autograd.grad(out["target"].sum(), [xg[m] for m in MODALITIES],
60                 retain_graph=False)
61             gnorm = torch.stack([grads[i].detach().norm(dim=-1) for i in range(3)], dim=1) # (B
62                 ,3,L)
63             B, _, L = gnorm.shape
64             alpha = out.get("alpha")
65             if alpha is not None and alpha.size(0) == B and alpha.size(1) == 3 * L:
66                 w_att = alpha.detach().reshape(B, 3, L)
67             else:
68                 # 兜底: 无法取注意力时退化为梯度幅值
69                 w_att = gnorm.clone()
70             w_agrad = w_att * gnorm
71             return (w_ig.detach().cpu().numpy(), w_att.cpu().numpy(), w_agrad.detach().cpu()).

```

```

        numpy())
66
67
68 def main():
69     ap = argparse.ArgumentParser()
70     ap.add_argument("--pkl", default=None)
71     ap.add_argument("--split", default=None)
72     ap.add_argument("--name", default="att4")
73     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
74     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
75     ap.add_argument("--steps", type=int, default=20)
76     ap.add_argument("--baseline", default="zero", choices=["zero", "mean"])
77     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
78     ap.add_argument("--batch_size", type=int, default=8)
79     ap.add_argument("--limit", type=int, default=0)
80     ap.add_argument("--device", default="cpu")
81     a = ap.parse_args()
82
83     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
84     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
85     split, key = load_split(pkl, stats, a.split)
86     if a.limit:
87         from data_utils import slice_split
88         split = slice_split(split, a.limit)
89     N = split["N"]
90     L = split["L"]["text"]
91     print("[DATA] %s/%s N=%d L=%d | IG steps=%d baseline=%s target=%s"
92           % (os.path.basename(pkl), key, N, L, a.steps, a.baseline, a.target))
93
94     W_IG = np.zeros((N, 3, L), np.float64)
95     W_AT = np.zeros((N, 3, L), np.float64)
96     W_AG = np.zeros((N, 3, L), np.float64)
97     for i0 in range(0, N, a.batch_size):
98         idxs = list(range(i0, min(i0 + a.batch_size, N)))
99         b = make_batch(split, idxs, a.device)
100         ig, att, ag = ig_batch(models, b, a.steps, a.baseline, a.target)
101         W_IG[i0:i0 + len(idxs)] = ig
102         W_AT[i0:i0 + len(idxs)] = att
103         W_AG[i0:i0 + len(idxs)] = ag
104         if (i0 // a.batch_size) % 20 == 0:
105             print(" %d/%d" % (min(i0 + a.batch_size, N), N))
106
107     # 有效位掩码（重要性只在有效位内有意义）
108     valid = np.zeros((N, 3, L), bool)
109     for i in range(N):
110         for k, m in enumerate(MODALITIES):

```

```

111         valid[i, k, :int((~split["pad"])[m][i]).sum())] = True
112     for W in (W_IG, W_AT, W_AG):
113         W[~valid] = 0.0
114     path = save_npz(os.path.join(a.out_dir, "q3_temporal_ig_%s.npz" % a.name),
115                     ids=np.asarray(split["ids"]).astype(str),
116                     w_ig=W_IG, w_att=W_AT, w_agrad=W_AG, valid=valid,
117                     meta=np.array([a.steps, a.baseline, a.target], dtype=object))
118     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,
119                             dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N,
120                                 steps=a.steps, baseline=a.baseline, target=a.target)),
121               w_ig_mean=[round(float(W_IG[:, k].sum(1).mean()), 4) for k in range(3)],
122               w_att_mean=[round(float(W_AT[:, k].sum(1).mean()), 4) for k in range(3)],
123               sparsity_ig_top5=[round(float(np.sort(W_IG[i, k, valid[i, k]]][::-1][:5].
124               sum() /
125               max(W_IG[i, k, valid[i, k]].sum(), 1e-9)),
126               3)
127               for i in range(min(N, 20)) for k in range(1)],
128               data=os.path.relpath(path, ROOT))
129     save_json(os.path.join(a.out_dir, "q3_temporal_ig_%s_report.json" % a.name), rep)
130     print("[SAVE] %s" % os.path.relpath(path, ROOT))
131     print("IG_GRADCAM_OK")
132     return 0
133 if __name__ == "__main__":
134     sys.exit(main())

```

C.7 问题三：遮挡重要性（窗宽 3 滑窗）

code/03_xai/occlusion.py（共 116 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S3: 模型无关的遮挡重要性（作为解释的验证基准）。
3
4  定义：把模态 m 的窗口 [start, start+w) 用**相邻段**替换（保持序列长度与形态，
5  避免“删掉一段导致长度变短”引入的伪影），测量目标标量的变化
6       $d_k = t(X) - t(X - \text{window}_k)$ ,
7  位置 k 的重要性取覆盖它的所有窗口的 |d| 均值。
8
9  为什么用相邻段替换而不是置零：置零在归一化空间里等价于“该段取训练均值”，
10  会同时改变“语义内容”与“是否存在信息”两件事；相邻段替换只改变内容，
11  更接近“如果这段时间说的是别的内容”的反事实，且对输出的扰动量级可解释。
12
13  用法：
14  python occlusion.py --pk1 <pk1> --name att4 --width 3 --stride 1 --out_dir <runs/q3>
15  python occlusion.py --pk1 ../附件2\\aligned_50.pk1 --split valid --name valid --stride 2

```

```

16 """
17 import argparse
18 import os
19 import sys
20
21 import numpy as np
22 import torch
23
24 HERE = os.path.dirname(os.path.abspath(__file__))
25 if HERE not in sys.path:
26     sys.path.append(HERE)
27 from xai_common import (MODALITIES, load_backbone, load_split, make_batch, # noqa: E402
28                          predict_batch, occlude_window, save_npz, save_json,
29                          q3_meta, ROOT)
30
31
32 def occl_batch(models, b, width=3, stride=1, target="mu"):
33     """返回 (w_occ (B,3,L), d_win 列表, 窗口起点列表)。"""
34     B = b["x"]["text"].size(0)
35     L = b["x"]["text"].size(1)
36     base = predict_batch(models, b, target=target)["target"].detach()
37     W = np.zeros((B, 3, L), np.float64)
38     CNT = np.zeros((B, 3, L), np.float64)
39     d_all = {}
40     starts_all = {}
41     for k, m in enumerate(MODALITIES):
42         nvalid = int((~(b["pad"][m][0])).sum().item()) if B else L
43         starts = list(range(0, max(1, nvalid - width + 1), stride))
44         if not starts:
45             starts = [0]
46         starts_all[m] = starts
47         d = np.zeros((B, len(starts)), np.float64)
48         for j, st in enumerate(starts):
49             bo = occlude_window(b, m, st, width)
50             t = predict_batch(models, bo, target=target)["target"].detach()
51             dd = (base - t).cpu().numpy()
52             d[:, j] = dd
53             for r in range(B):
54                 e = min(L, st + width)
55                 W[r, k, st:e] += np.abs(dd[r])
56                 CNT[r, k, st:e] += 1.0
57         d_all[m] = d
58     W = W / np.maximum(CNT, 1.0)
59     return W, d_all, starts_all
60
61

```

```

62 def main():
63     ap = argparse.ArgumentParser()
64     ap.add_argument("--pkl", default=None)
65     ap.add_argument("--split", default=None)
66     ap.add_argument("--name", default="att4")
67     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
68     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
69     ap.add_argument("--width", type=int, default=3)
70     ap.add_argument("--stride", type=int, default=1)
71     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
72     ap.add_argument("--batch_size", type=int, default=16)
73     ap.add_argument("--limit", type=int, default=0)
74     ap.add_argument("--device", default="cpu")
75     a = ap.parse_args()
76
77     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
78     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
79     split, key = load_split(pkl, stats, a.split)
80     if a.limit:
81         from data_utils import slice_split
82         split = slice_split(split, a.limit)
83     N, L = split["N"], split["L"]["text"]
84     print("[DATA] %s/%s N=%d L=%d | 窗口 w=%d 步长 %d target=%s"
85           % (os.path.basename(pkl), key, N, L, a.width, a.stride, a.target))
86
87     W = np.zeros((N, 3, L), np.float64)
88     for i0 in range(0, N, a.batch_size):
89         idxs = list(range(i0, min(i0 + a.batch_size, N)))
90         b = make_batch(split, idxs, a.device)
91         w, _, _ = occl_batch(models, b, a.width, a.stride, a.target)
92         W[i0:i0 + len(idxs)] = w
93         if (i0 // a.batch_size) % 20 == 0:
94             print(" %d/%d" % (min(i0 + a.batch_size, N), N))
95
96     valid = np.zeros((N, 3, L), bool)
97     for i in range(N):
98         for k, m in enumerate(MODALITIES):
99             valid[i, k, :int((~split["pad"])[m][i]).sum())] = True
100     W[~valid] = 0.0
101     path = save_npz(os.path.join(a.out_dir, "q3_temporal_occ_%s.npz" % a.name),
102                    ids=np.asarray(split["ids"]).astype(str), w_occ=W, valid=valid,
103                    meta=np.array([a.width, a.stride, a.target], dtype=object))
104     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,
105                            dict(pk1=os.path.relpath(pkl, ROOT), split=key, n=N,
106                                width=a.width, stride=a.stride, target=a.target)),
107               w_occ_sum_mean=[round(float(W[:, k].sum(1).mean()), 4) for k in range(3)],

```

```

108         data=os.path.relpath(path, ROOT))
109     save_json(os.path.join(a.out_dir, "q3_temporal_occ_%s_report.json" % a.name), rep)
110     print("[SAVE] %s" % os.path.relpath(path, ROOT))
111     print("OCCLUSION_OK")
112     return 0
113
114
115 if __name__ == "__main__":
116     sys.exit(main())

```

C.8 问题三：三源融合与 γ 单纯形约束

code/03_xai/temporal_fuse.py (共 152 行)

```

1  #-*- coding: utf-8 -*-
2  r"""S4-a: 三源时间重要性的归一化与融合（gamma 在验证集上按与遮挡源的一致性标定）。
3
4  三源:
5  w_att 模型内生注意力（基线，已知”注意力!=重要性”）
6  w_ig 积分梯度（公理化）
7  w_occ 遮挡（模型无关，作为验证基准）
8
9  融合：先对每个样本、每个模态在**有效位内**做 min-max 归一化，再线性加权
10     w = (gamma1.w~_att + gamma2.w~_ig + gamma3.w~_occ) / (gamma1+gamma2+gamma3)，最后再归一化一次。
11  gamma 的标定：在单纯形上枚举（**约束每个 gamma >= 0.1**，避免退化为”只留遮挡源”这种平凡解），
12     以”融合结果与遮挡源的 Spearman 一致性（跨样本、跨模态、有效位内聚合）”为准则；
13     同时报告等权 (1,1,1) 对照。
14
15  用法:
16  python temporal_fuse.py --name valid --out_dir ../runs/q3
17  """
18  import argparse
19  import itertools
20  import json
21  import os
22  import sys
23
24  import numpy as np
25
26  HERE = os.path.dirname(os.path.abspath(__file__))
27  if HERE not in sys.path:
28      sys.path.append(HERE)
29  from xai_common import MODALITIES, save_npz, save_json, ROOT # noqa: E402
30
31  SOURCES = ("att", "ig", "occ")
32

```

```

33
34 def load_sources(out_dir, name):
35     ig = np.load(os.path.join(out_dir, "q3_temporal_ig_%s.npz" % name), allow_pickle=True)
36     oc = np.load(os.path.join(out_dir, "q3_temporal_occ_%s.npz" % name), allow_pickle=True)
37     w = dict(att=ig["w_att"], ig=ig["w_ig"], occ=oc["w_occ"])
38     return w, ig["valid"], np.asarray(ig["ids"]).astype(str)
39
40
41 def norm_valid_mat(W, valid):
42     """对 (N,3,L) 逐样本逐模态在有效位内 min-max 归一化（无效位保持 0）。"""
43     A = np.zeros_like(W, dtype=np.float64)
44     N, K, L = W.shape
45     for i in range(N):
46         for k in range(K):
47             v = valid[i, k]
48             if not v.any():
49                 continue
50             seg = W[i, k, v].astype(np.float64)
51             lo, hi = seg.min(), seg.max()
52             if hi - lo < 1e-12:
53                 continue
54             A[i, k, v] = (seg - lo) / (hi - lo)
55     return A
56
57
58 def spearman(a, b):
59     from scipy.stats import spearmanr
60     if a.size < 3 or np.std(a) < 1e-12 or np.std(b) < 1e-12:
61         return np.nan
62     return float(spearmanr(a, b).statistic)
63
64
65 def agreement(W, ref, valid):
66     """跨样本/跨模态/有效位聚合的 Spearman 一致性。"""
67     a, b = [], []
68     for i in range(W.shape[0]):
69         for k in range(W.shape[1]):
70             v = valid[i, k]
71             if v.sum() < 3:
72                 continue
73             a.append(W[i, k, v])
74             b.append(ref[i, k, v])
75     if not a:
76         return np.nan
77     return spearman(np.concatenate(a), np.concatenate(b))
78

```

```

79
80 def simplex_grid(step=0.1, min_g=0.1):
81     """枚举 gamma1+gamma2+gamma3=1, 每个 >= min_g."""
82     out = []
83     n = int(round(1.0 / step))
84     for i in range(n + 1):
85         for j in range(n + 1 - i):
86             k = n - i - j
87             g = np.array([i, j, k]) * step
88             if g.min() >= min_g - 1e-9 and abs(g.sum() - 1.0) < 1e-9:
89                 out.append(tuple(np.round(g, 3)))
90     return sorted(set(out))
91
92
93 def fuse(ws, gamma):
94     g = np.asarray(gamma, dtype=np.float64)
95     g = g / g.sum()
96     W = g[0] * ws["att"] + g[1] * ws["ig"] + g[2] * ws["occ"]
97     return W
98
99
100 def main():
101     ap = argparse.ArgumentParser()
102     ap.add_argument("--name", default="valid")
103     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
104     ap.add_argument("--step", type=float, default=0.1)
105     a = ap.parse_args()
106
107     W_raw, valid, ids = load_sources(a.out_dir, a.name)
108     N, K, L = W_raw["occ"].shape
109     print("[DATA] %s N=%d 三源形状=(%d,%d,%d)" % (a.name, N, N, K, L))
110     ws = {k: norm_valid_mat(W_raw[k], valid) for k in SOURCES}
111     for k in SOURCES:
112         print(" [SRC] %-4s 归一化后均值 文/语/视 = %.4f / %.4f / %.4f"
113               % (k, *[ws[k][:, i][valid[:, i]].mean() for i in range(3)]))
114
115     print(" [gamma] 单纯形枚举 (每源 >= %.1f), 以与遮挡源的一致性为准则: " % 0.1)
116     rows = []
117     for g in simplex_grid(a.step):
118         W = fuse(ws, g)
119         ag = agreement(norm_valid_mat(W, valid), ws["occ"], valid)
120         rows.append((g, ag))
121     rows_sorted = sorted(rows, key=lambda t: -(t[1] if np.isfinite(t[1]) else -9))
122     for g, ag in rows_sorted[:5]:
123         print("    gamma=(%.1f, %.1f, %.1f) 一致性 rho=%.4f" % (*g, ag))
124     g_best = rows_sorted[0][0]

```

```

125     g_eq = (1 / 3, 1 / 3, 1 / 3)
126     W_best = fuse(Ws, g_best)
127     W_eq = fuse(Ws, g_eq)
128     print(" [gamma] 选定 gamma_att/gamma_ig/gamma_occ = (%.2f, %.2f, %.2f) (等权对照 rho=%.4f) "
129           % (*g_best, agreement(norm_valid_mat(W_eq, valid), Ws["occ"], valid)))
130
131     Wb = norm_valid_mat(W_best, valid)
132     We = norm_valid_mat(W_eq, valid)
133     path = save_npz(os.path.join(a.out_dir, "q3_temporal_fused_%.s.npz" % a.name),
134                     ids=ids, valid=valid, w_fused=Wb, w_fused_equal=We,
135                     w_att=Ws["att"], w_ig=Ws["ig"], w_occ=Ws["occ"],
136                     gamma=np.array(g_best))
137     rep = dict(name=a.name, n=N, gamma=list(g_best), gamma_equal=list(g_eq),
138               grid=[dict(gamma=list(g), spearman_occ=(None if not np.isfinite(ag) else
139                    round(ag, 4)))
140                     for g, ag in rows_sorted],
141               source_mean={k: [round(float(Ws[k][:, i][valid[:, i]].mean()), 4)
142                               for i in range(3)] for k in SOURCES},
143               agreement_with_occ={k: round(agreement(Ws[k], Ws["occ"], valid), 4)
144                                   for k in SOURCES},
145               data=os.path.relpath(path, ROOT))
146     save_json(os.path.join(a.out_dir, "q3_gamma_%.s.json" % a.name), rep)
147     print("[SAVE] %s" % os.path.relpath(path, ROOT))
148     print("FUSE_OK")
149     return 0
150
151 if __name__ == "__main__":
152     sys.exit(main())

```

C.9 问题三：证据定位（词元对齐、语音时段映射、关键帧抽取）

code/03_xai/evidence.py（共 326 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S5：证据定位（文本片段 / 语音时段 / 视觉关键帧）——题目要求“可回看原始素材”。
3
4  链路：
5  1. 文本：‘text_bert’ 的词元编号 + ‘vocab.txt’（行号即 id）解码 WordPiece -> 合并 ‘##’ 续接
6     -> 得到“词”序列及其**词元位置集合**（词元位置 = 对齐段号，见 section1.2 的对齐粒度实测）
7     -> 用 ‘difflib’ 把解码词对齐回原始 ‘raw_text’ 的字符区间（失败显式标记，不静默降级）；
8  2. 语音：高重要度的段号合并为连续区间，按比例映射为时间（附件2 不提供逐词时间戳，
9     声明为近似）并用视频音频能量曲线佐证；
10 3. 视觉：高重要度段取段内“视觉活跃度最大”的帧作为关键帧，用 ffmpeg 抽帧存盘。
11
12 输出：

```

```

13 submission/evidence/{id}_kf.jpg      关键帧
14 runs/q3/q3_evidence_{name}.json      逐样本证据记录（含字符区间、秒区间、帧时刻）
15 runs/q3/q3_audio_energy_{name}.npz   各样本音频能量曲线（论文解释卡用）
16
17 用法:
18 python evidence.py --name att4 --out_dir ../../runs/q3
19 """
20 import argparse
21 import difflib
22 import json
23 import os
24 import re
25 import subprocess
26 import sys
27 import wave
28
29 import numpy as np
30
31 HERE = os.path.dirname(os.path.abspath(__file__))
32 CODE = os.path.dirname(HERE)
33 ROOT = os.path.dirname(CODE)
34 if HERE not in sys.path:
35     sys.path.append(HERE)
36 from xai_common import MODALITIES, save_json, ROOT as W_ROOT      # noqa: E402
37 from data_utils import load_pickle                                # noqa: E402
38
39 VOCAB = os.path.join(CODE, "assets", "bert-base-uncased_vocab.txt")
40 WORD_RE = re.compile(r"[A-Za-z0-9]+(?:[' \-][A-Za-z0-9]+)*")
41
42
43 # -----
44 def load_vocab(path=VOCAB):
45     if not os.path.isfile(path):
46         raise SystemExit("缺少词表 %s, 请先运行 fetch_assets.py" % path)
47     with open(path, encoding="utf-8") as f:
48         return [l.rstrip("\n") for l in f]
49
50
51 def decode_words(ids, mask, vocab, merge_punct=True):
52     """词元序列 -> [(word, [token_pos, ...])], 跳过 [CLS]/[SEP]/填充。
53
54     merge_punct=True 时把纯标点词元（如 “'”、“-”、“,”）并入相邻词，
55     使解码词序列与原始转写的词序列更接近（实测可把对齐率由 ~0.75 提升到 ~0.95）。
56
57     """
58     words, cur, positions = [], None, []
59     for p, (i, m) in enumerate(zip(ids, mask)):

```

```

59     if m == 0:
60         break
61     tok = vocab[int(i)] if 0 <= int(i) < len(vocab) else "[UNK]"
62     if tok in ("[CLS]", "[SEP]"):
63         continue
64     if tok.startswith("##"):
65         if cur is None:
66             cur, positions = tok[2:], [p]
67         else:
68             cur, positions = cur + tok[2:], positions + [p]
69     else:
70         if cur is not None:
71             words.append((cur, positions))
72             cur, positions = tok, [p]
73     if cur is not None:
74         words.append((cur, positions))
75     if not merge_punct:
76         return words
77     # 合并纯标点词元：与前一单词拼合（无前词则与后一单词拼合）
78     merged = []
79     for w, poss in words:
80         if w and all(not ch.isalnum() for ch in w):
81             if merged:
82                 pw, pposs = merged[-1]
83                 merged[-1] = (pw + w, pposs + poss)
84                 continue
85             if (not merged) and w in ("'", "' ", "\'", "' ", "' '"):
86                 merged.append((w, poss))
87                 continue
88             merged.append((w, poss))
89     # 后处理：相邻的"词+纯标点"再合并一次（处理句首标点情形）
90     out = []
91     i = 0
92     while i < len(merged):
93         w, poss = merged[i]
94         if (i + 1 < len(merged) and merged[i + 1][0] and
95             all(not ch.isalnum() for ch in merged[i + 1][0]) and len(w) > 1):
96             out.append((w + merged[i + 1][0], poss + merged[i + 1][1]))
97             i += 2
98             continue
99         out.append((w, poss))
100         i += 1
101     return out
102
103
104 def align_to_raw(words, raw_text):

```

```

105     """把解码词对齐回 raw_text 的字符区间。
106
107     返回 (spans, info): spans 为逐词的 (start, end) (未对齐项为 None);
108     info 记录对齐率, 供论文如实报告”对齐失败样本数”。
109     """
110     spans = [None] * len(words)
111     raw_words = [(m.group(0), m.start(), m.end()) for m in WORD_RE.finditer(raw_text)]
112     a = [w.lower() for w, _ in words]
113     b = [w.lower() for w, _, _ in raw_words]
114     sm = difflib.SequenceMatcher(a=a, b=b, autojunk=False)
115     hit = 0
116     for blk in sm.get_matching_blocks():
117         for t in range(blk.size):
118             spans[blk.a + t] = (raw_words[blk.b + t][1], raw_words[blk.b + t][2])
119             hit += 1
120     info = dict(n_word=len(words), n_raw=len(raw_words), matched=hit,
121                ratio=round(float(hit / max(len(words), 1)), 3))
122     return spans, info
123
124
125 def merge_runs(pos, gap=1):
126     """把位置列表合并为连续区间 [start, end)。"""
127     pos = sorted(set(int(p) for p in pos))
128     if not pos:
129         return []
130     runs, s, e = [], pos[0], pos[0] + 1
131     for p in pos[1:]:
132         if p <= e + gap - 1:
133             e = p + 1
134         else:
135             runs.append([s, e])
136             s, e = p, p + 1
137     runs.append([s, e])
138     return runs
139
140
141 def seg_to_seconds(seg_start, seg_end, n_valid, duration):
142     """段号区间 -> 秒区间 (比例映射; duration 为该样本视频时长)。"""
143     if not duration or n_valid <= 0:
144         return None, None
145     per = float(duration) / float(n_valid)
146     return round(seg_start * per, 2), round(min(seg_end, n_valid) * per, 2)
147
148
149 # -----
150 def ffmpeg_exe():

```

```

151 import imageio_ffmpeg
152 return imageio_ffmpeg.get_ffmpeg_exe()
153
154
155 def grab_frame(video, t_sec, out_jpg, width=320, q=5):
156     os.makedirs(os.path.dirname(out_jpg), exist_ok=True)
157     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-ss", "%.3f" % max(0.0, t_sec),
158           "-i", video, "-frames:v", "1", "-vf", "scale=%d:-1" % width,
159           "-q:v", str(q), out_jpg]
160     r = subprocess.run(cmd, capture_output=True)
161     return os.path.isfile(out_jpg) and os.path.getsize(out_jpg) > 0, (r.stderr or b"")
162     [-300:]
163
164 def audio_energy(video, tmp_wav, hop=0.05):
165     """抽出 16k 单声道 wav 并算短时能量曲线（返回 times, rms）。"""
166     os.makedirs(os.path.dirname(tmp_wav), exist_ok=True)
167     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-i", video, "-vn",
168           "-ac", "1", "-ar", "16000", tmp_wav]
169     r = subprocess.run(cmd, capture_output=True)
170     if not os.path.isfile(tmp_wav):
171         return None, None, (r.stderr or b"")[-300:]
172     with wave.open(tmp_wav, "rb") as w:
173         sr = w.getframerate()
174         n = w.getnframes()
175         data = np.frombuffer(w.readframes(n), dtype=np.int16).astype(np.float32) /
176         32768.0
177         step = max(1, int(hop * sr))
178         nseg = max(1, len(data) // step)
179         rms = np.sqrt(np.mean(data[:nseg * step].reshape(nseg, step) ** 2, axis=1))
180         times = (np.arange(nseg) + 0.5) * step / sr
181         return times, rms, None
182
183 # -----
184 def main():
185     ap = argparse.ArgumentParser()
186     ap.add_argument("--name", default="att4")
187     ap.add_argument("--out_dir", default=os.path.join(W_ROOT, "runs", "q3"))
188     ap.add_argument("--sub_dir", default=os.path.join(W_ROOT, "submission"))
189     ap.add_argument("--meta", default=os.path.join(W_ROOT, "data_att", "att_meta.json"))
190     ap.add_argument("--pkl", default=os.path.join(W_ROOT, "data_att", "att4_aligned.pkl"),
191                     help="缓存 pkl（含 text_bert 与 raw_text，用于文本证据解码）")
192     ap.add_argument("--top_seg", type=int, default=3)
193     ap.add_argument("--top_word", type=int, default=3)
194     ap.add_argument("--phrase_win", type=int, default=2)

```

```

195 ap.add_argument("--no_video", action="store_true", help="不抽帧/不提音频（只算文本与时段）")
196 a = ap.parse_args()
197
198 fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_%.npz" % a.name),
199                 allow_pickle=True)
200 W = fused["w_fused"]
201 valid = fused["valid"]
202 ids = [str(x) for x in fused["ids"]]
203 meta = json.load(open(a.meta, encoding="utf-8"))
204 recs = {r["id"]: r for r in meta[a.name]}
205 vocab = load_vocab()
206
207 # 文本证据需要 text_bert（词元编号）与 raw_text：从重建后的缓存 pkl 读取
208 tb_all, rt_all, idx_map = None, None, {}
209 if a.pkl and os.path.isfile(a.pkl):
210     d = load_pickle(a.pkl)
211     if "id" in d:
212         pkl_ids = [str(x) for x in np.asarray(d["id"]).reshape(-1)]
213         idx_map = {sid: i for i, sid in enumerate(pkl_ids)}
214     if "text_bert" in d:
215         tb_all = np.asarray(d["text_bert"])
216     if "raw_text" in d:
217         rt_all = [str(x) for x in np.asarray(d["raw_text"]).reshape(-1)]
218     print(" [PKL] %s: text_bert=%s raw_text=%s"
219           % (os.path.basename(a.pkl),
220              "有" if tb_all is not None else "无",
221              "有" if rt_all is not None else "无"))
222
223 ev_dir = os.path.join(a.sub_dir, "evidence")
224 out, energies = {}, {}
225 align_bad = []
226 for i, sid in enumerate(ids):
227     r = recs.get(sid, {})
228     k = idx_map.get(sid, None)
229     raw_text = (rt_all[k] if (rt_all is not None and k is not None and k < len(rt_all)
230 ))
231
232         else (r.get("raw_text", "") or ""))
233     dur = r.get("duration_s")
234     tb = tb_all[k] if (tb_all is not None and k is not None and k < len(tb_all)) else
235     None
236     n_text = int(valid[i, 0].sum())
237     n_audio = int(valid[i, 1].sum())
238     n_vision = int(valid[i, 2].sum())
239     rec = dict(id=sid)
240
241     # —— 文本证据 ——

```

```

239 w_text = np.asarray(W[i, 0], np.float64)
240 words = []
241 if tb is not None:
242     tb = np.asarray(tb)
243     words = decode_words(tb[0], tb[1], vocab)
244 if words:
245     spans, al = align_to_raw(words, raw_text)
246     if al["ratio"] < 0.8:
247         align_bad.append(dict(id=sid, **al))
248     ww = []
249     for (word, poss), sp in zip(words, spans):
250         ww.append((word, poss, sp, float(max(w_text[p] for p in poss if p < len(
w_text)))))
251     ww_sorted = sorted(ww, key=lambda t: -t[3])
252     picks = [t for t in ww_sorted[:max(1, a.top_word)]]
253     pick_idx = [ww.index(t) for t in picks]
254     lo = max(0, min(pick_idx) - a.phrase_win)
255     hi = min(len(ww) - 1, max(pick_idx) + a.phrase_win)
256     phrase_words = [t[0] for t in ww[lo:hi + 1]]
257     phrase = "".join(phrase_words)
258     cspans = [t[2] for t in ww[lo:hi + 1] if t[2]]
259     segs = merge_runs([p for t in picks for p in t[1]])
260     rec["text_evidence"] = dict(
261         top_words=[dict(word=t[0], seg=t[1], char_span=t[2], w=round(t[3], 4))
262                     for t in picks],
263         phrase=phrase,
264         phrase_char_span=(min(s[0] for s in cspans), max(s[1] for s in cspans))
265                           if cspans else None),
266         seg_runs=segs, n_valid=n_text, align=al)
267 else:
268     rec["text_evidence"] = dict(note="无 text_bert (无法解码), 仅给段号",
269                                seg_runs=[[int(x) for x in rn] for rn in
270                                             merge_runs(np.argsort(-w_text[:n_text])
271 [:a.top_seg])],
272                                n_valid=n_text)
273
274 # —— 语音证据 ——
275 w_audio = np.asarray(W[i, 1], np.float64)
276 top_a = [int(x) for x in np.argsort(-w_audio[:n_audio])[:a.top_seg]]
277 runs_a = merge_runs(top_a)
278 s0, s1 = (seg_to_seconds(runs_a[0][0], runs_a[-1][1], n_audio, dur)
279           if runs_a else (None, None))
280 rec["audio_evidence"] = dict(top_segs=top_a, seg_runs=runs_a,
281                               start_s=s0, end_s=s1, n_valid=n_audio, duration_s=
dur)
282

```

```

282 # —— 视觉证据 ——
283 w_vision = np.asarray(W[i, 2], np.float64)
284 top_v = [int(x) for x in np.argsort(-w_vision[:n_vision])[a.top_seg]]
285 runs_v = merge_runs(top_v)
286 t0, t1 = (seg_to_seconds(runs_v[0][0], runs_v[0][1], n_vision, dur)
287           if runs_v else (None, None))
288 kf_t = None if t0 is None else round((t0 + (t1 if t1 else t0)) / 2.0, 2)
289 rec["vision_evidence"] = dict(top_segs=top_v, seg_runs=runs_v,
290                               keyframe_time_s=kf_t, n_valid=n_vision)
291 if (not a.no_video) and r.get("video") and kf_t is not None:
292     out_jpg = os.path.join(ev_dir, "%s_kf.jpg" % sid)
293     ok, err = grab_frame(r["video"], kf_t, out_jpg)
294     rec["vision_evidence"]["keyframe_file"] = os.path.relpath(out_jpg, W_ROOT)
295     rec["vision_evidence"]["keyframe_ok"] = bool(ok)
296     if not ok:
297         rec["vision_evidence"]["keyframe_err"] = err.decode("utf-8", "replace")
298     if sid not in energies:
299         tw = os.path.join(a.out_dir, "_audio", "%s.wav" % sid)
300         tt, rr, err2 = audio_energy(r["video"], tw)
301         if tt is not None:
302             energies[sid] = dict(t=tt, rms=rr)
303             np.savez_compressed(os.path.join(a.out_dir, "_audio", "%s_energy.npz" %
304 sid),
305                               t=tt, rms=rr)
306         else:
307             rec["vision_evidence"]["audio_err"] = str(err2)[:200]
308         try:
309             os.remove(tw)
310         except OSError:
311             pass
312     out[sid] = rec
313     if (i + 1) % 5 == 0 or i + 1 == len(ids):
314         print(" %d/%d" % (i + 1, len(ids)))
315
316 save_json(os.path.join(a.out_dir, "q3_evidence_%s.json" % a.name), out)
317 if align_bad:
318     print(" [警告] 文本对齐率 <0.8 的样本 %d 条: %s" % (len(align_bad), align_bad[:3]))
319     print(" [KF] 关键帧 %d 张; 音频能量曲线 %d 条"
320           % (len([1 for v in out.values() if v.get("vision_evidence", {}).get("keyframe_ok")]),
321              len(energies)))
322     print("EVIDENCE_OK")
323     return 0
324
325 if __name__ == "__main__":
326     sys.exit(main())

```

C.10 问题三：并联证据头训练（骨干冻结、忠实性正则）

code/03_xai/train_evidence_head.py（共 205 行）

```
1  # -*- coding: utf-8 -*-
2  r"""D4/D9: 可解释性证据头（冻结骨干 + 并联解释头 + 解释性正则训练）。
3
4  设计（论文 section5.3“可解释性模型的网络结构、目标函数、训练方案与关键参数”）：
5  骨干**完全冻结**并保持预测通路不变；证据头挂在其**编码器输出**（逐模态 50x d 表示）上，
6  只用于产生解释： $s^m_k = \text{softmax}_k(w_m^T \tanh(W_m h^m_k + b_m))$ ，
7  模态作用度  $\pi = \text{softmax}_m(u^T \tanh(V h^m))$ ， $h^m = \text{Sigma}_k s^m_k h^m_k$ 。
8
9  目标函数（全部可微；用“软删除/软保留”把忠实性写成可训练的损失）：
10  $L = \alpha_f [-(t(X) - t(X; \text{miss}=s))^2 + (t(X) - t(X; \text{miss}=1-s))^2]$ 
11  $+ \alpha_1 \text{Sigma}_m \|s^m\|_1 + \alpha_2 \text{Sigma}_m \text{TV}(s^m) + \alpha_3 \text{Sigma}_m (\pi_m - \pi_m^{\text{Shapley}})^2$ 
12
13 第一项：按 s 软删除后的预测变化越大越好（**忠实性**）；
14 第二项：只保留 s 的软权重时的预测应接近全模态（**充分性**）；
15 第三项/第四项：稀疏性与时间平滑；第五项（可选）：与精确 Shapley 的模态作用度对齐。
16
17 训练方案：只训证据头（骨干冻结、eval 模式），AdamW，lr 1e-3，30 轮，按验证集
18 “忠实性 - 充分性偏差”加权目标早停；关键参数 alpha 在验证集上网格择优（默认给定值）。
19
20 用法：
21 python train_evidence_head.py --epochs 12 --out ../\runs\q3
22 """
23 import argparse
24 import json
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30 import torch.nn as nn
31 import torch.nn.functional as F
32
33 HERE = os.path.dirname(os.path.abspath(__file__))
34 if HERE not in sys.path:
35     sys.path.append(HERE)
36 from xai_common import (MODALITIES, load_backbone, load_split, make_batch, # noqa: E402
37                          predict_batch, save_json, q3_meta, ROOT)
38
39
40 class EvidenceHead(nn.Module):
41     """逐模态时间证据头 + 模态门控（并联，不接入预测通路）。"""
42
43     def __init__(self, hidden, n_mod=3):
```

```

44     super().__init__()
45     self.hidden = hidden
46     self.W = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in MODALITIES})
47     self.w = nn.ParameterDict({m: nn.Parameter(torch.zeros(hidden)) for m in
MODALITIES})
48     self.V = nn.Linear(hidden, hidden)
49     self.u = nn.Linear(hidden, 1)
50
51     def forward(self, h, pad):
52         """h: 模态->(B,L,H); pad: 模态->(B,L); 返回 s (B,3,L) 、 pi (B,3) 。”"""
53         S, pi_logit = [], []
54         for m in MODALITIES:
55             z = torch.tanh(self.W[m](h[m])) # (B,L,H)
56             sc = torch.einsum("blh,h->bl", z, self.w[m]) # (B,L)
57             sc = sc.masked_fill(pad[m], -1e9)
58             s = torch.softmax(sc, dim=1)
59             S.append(s)
60             hbar = (s.unsqueeze(-1) * h[m]).sum(dim=1) # (B,H)
61             pi_logit.append(self.u(torch.tanh(self.V(hbar))).squeeze(-1))
62         S = torch.stack(S, dim=1) # (B,3,L)
63         pi = torch.softmax(torch.stack(pi_logit, dim=1), dim=1)
64         return S, pi
65
66
67     def capture(mods, models):
68         """抓取骨干各模态编码器输出 h[m] 的 hook (兼容 [(model, ckpt), ...] 与 [model, ...]) 。”"""
69         store, handles = {}, []
70         m0 = models[0][0] if isinstance(models[0], tuple) else models[0]
71         for m in mods:
72             def mk(name):
73                 def hook(module, inp, out):
74                     store[name] = out
75                 return hook
76             handles.append(m0.enc[m].register_forward_hook(mk(m)))
77         return store, handles
78
79
80     def forward_with_miss(models, b, miss_override):
81         """用给定的 miss (可为软值) 跑集成前向, 返回 (score, prob). x 同步按软掩码缩放。”"""
82         bb = dict(x={}, miss={}, pad={})
83         for m in MODALITIES:
84             bb["pad"][m] = b["pad"][m]
85             bb["miss"][m] = miss_override[m]
86             bad = (miss_override[m] + b["pad"][m].float()).clamp(0, 1).unsqueeze(-1)
87             bb["x"][m] = b["x"][m] * (1.0 - bad)
88         out = predict_batch(models, bb, target="mu")

```

```

89     return out["score"], out["prob"]
90
91
92 def tv(s):
93     return (s[:, :, 1:] - s[:, :, :-1]).abs().mean()
94
95
96 def main():
97     ap = argparse.ArgumentParser()
98     ap.add_argument("--epochs", type=int, default=12)
99     ap.add_argument("--batch_size", type=int, default=64)
100    ap.add_argument("--lr", type=float, default=1e-3)
101    ap.add_argument("--alpha_f", type=float, default=1.0)
102    ap.add_argument("--alpha1", type=float, default=1e-3)
103    ap.add_argument("--alpha2", type=float, default=1e-3)
104    ap.add_argument("--alpha3", type=float, default=0.0,
105                    help="与 Shapley 对齐项权重 (>0 需先算训练集 Shapley)")
106    ap.add_argument("--limit", type=int, default=0)
107    ap.add_argument("--out", default=os.path.join(ROOT, "runs", "q3"))
108    ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
109    ap.add_argument("--device", default="cpu")
110    a = ap.parse_args()
111
112    models, stats, theta = load_backbone(a.ckpt_dir, a.device, verbose=False)
113    for m, _ in models:
114        m.eval()
115        for p in m.parameters():
116            p.requires_grad_(False)
117    tr, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "train")
118    va, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "valid")
119    if a.limit:
120        from data_utils import slice_split
121        tr = slice_split(tr, a.limit)
122    hidden = models[0][1]["cfg"]["hidden"]
123    print("[DATA] train N=%d valid N=%d hidden=%d" % (tr["N"], va["N"], hidden))
124
125    head = EvidenceHead(hidden).to(a.device)
126    opt = torch.optim.AdamW(head.parameters(), lr=a.lr, weight_decay=1e-4)
127    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=a.epochs)
128    hist = []
129    best = dict(score=-1e9, epoch=-1, state=None)
130
131    for ep in range(1, a.epochs + 1):
132        head.train()
133        tot, nb = 0.0, 0
134        perm = np.random.permutation(tr["N"])

```

```

135     for i0 in range(0, tr["N"], a.batch_size):
136         idxs = [int(i) for i in perm[i0:i0 + a.batch_size]]
137         b = make_batch(tr, idxs, a.device)
138         store, handles = capture(MODALITIES, models)
139         with torch.no_grad():
140             t_full, _ = forward_with_miss(models, b,
141                                           {m: b["miss"][m].float() for m in MODALITIES
142         })
143
144         for hd in handles:
145             hd.remove()
146             S, pi = head(store, b["pad"])
147             miss_soft = {m: (S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}
148             keep_soft = {m: (1.0 - S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}
149         }
150
151         t_del, _ = forward_with_miss(models, b, miss_soft)
152         t_keep, _ = forward_with_miss(models, b, keep_soft)
153         l_f = -((t_full - t_del) ** 2).mean() + ((t_full - t_keep) ** 2).mean()
154         l1 = sum(S[:, k].mean() for k in range(3)) / 3.0
155         l_tv = sum(tv(S[:, k:k + 1]) for k in range(3)) / 3.0
156         loss = a.alpha_f * l_f + a.alpha1 * l1 + a.alpha2 * l_tv
157         opt.zero_grad()
158         loss.backward()
159         nn.utils.clip_grad_norm_(head.parameters(), 1.0)
160         opt.step()
161         tot += float(loss.detach())
162         nb += 1
163     sched.step()
164     # —— 验证集评估：忠实性 - 充分性偏差（越大越好） ——
165     head.eval()
166     compr, suff = [], []
167     with torch.no_grad():
168         for i0 in range(0, min(va["N"], 256), 64):
169             idxs = list(range(i0, min(i0 + 64, va["N"])))
170             b = make_batch(va, idxs, a.device)
171             store, handles = capture(MODALITIES, models)
172             t_full, _ = forward_with_miss(models, b,
173                                           {m: b["miss"][m].float() for m in MODALITIES
174         })
175
176         for hd in handles:
177             hd.remove()
178             S, pi = head(store, b["pad"])
179             t_del, _ = forward_with_miss(models, b, {m: S[:, k] for k, m in enumerate(
180 (MODALITIES))})
181             t_keep, _ = forward_with_miss(models, b,
182                                           {m: (1 - S[:, k]) for k, m in enumerate(
183 MODALITIES))})

```

```

176         compr.append((t_full - t_del).abs().mean().item())
177         suff.append((t_full - t_keep).abs().mean().item())
178     c, s = float(np.mean(compr)), float(np.mean(suff))
179     sc = c - s
180     hist.append(dict(epoch=ep, loss=round(tot / max(nb, 1), 5), valid_compr=round(c,
181     5),
182                 valid_suff=round(s, 5), score=round(sc, 5)))
183     print(" [ep%02d] loss=%.5f valid 忠实性=%.5f 充分性偏差=%.5f 目标=%.5f"
184           % (ep, tot / max(nb, 1), c, s, sc))
185     if sc > best["score"]:
186         best = dict(score=sc, epoch=ep,
187                     state={k: v.detach().clone() for k, v in head.state_dict().items
188                     ()})
189     head.load_state_dict(best["state"])
190     torch.save(dict(state=head.state_dict(), cfg=dict(hidden=hidden, alpha_f=a.alpha_f,
191               alpha1=a.alpha1, alpha2=a.alpha2,
192               alpha3=a.alpha3, epochs=a.epochs,
193               lr=a.lr),
194               meta=q3_meta(a.ckpt_dir, models, theta, dict(train_n=tr["N"])),
195               history=hist, best_epoch=best["epoch"]),
196               os.path.join(a.out, "q3_evidence_head.pt"))
197     save_json(os.path.join(a.out, "q3_evidence_head_report.json"),
198               dict(best_epoch=best["epoch"], best_score=best["score"], history=hist,
199                   args=dict(alpha_f=a.alpha_f, alpha1=a.alpha1, alpha2=a.alpha2,
200                             alpha3=a.alpha3, epochs=a.epochs, lr=a.lr, batch=a.
201                             batch_size)))
202     print("[SAVE] runs/q3/q3_evidence_head.pt (最佳轮 %d, 目标 %.5f)" % (best["epoch"], best["score"]
203     ))
204     print("EVIDENCE_HEAD_OK")
205     return 0
206
207 if __name__ == "__main__":
208     sys.exit(main())

```

C.11 问题三：解释卡生成

code/03_xai/explain_card.py (共 201 行)

```

1  # -*- coding: utf-8 -*-
2  r"""S6-a: 典型样本解释卡 (JSON + 三联图)。
3
4  三联图内容 (论文 section5.3 典型样本解释卡):
5  (1) 三模态时间重要性条带 (含有效位边界与 top 片段标注);
6  (2) 文本词着色条 (按词级重要性深浅着色, 凸显关键证据词);
7  (3) 关键帧缩略图 + 音频能量曲线 (高亮证据时段)。

```

```

8
9 用法:
10 python explain_card.py --name att4 --out_dir ../../runs/q3 --fig_dir ../../figs
11 """
12 import argparse
13 import json
14 import os
15 import sys
16
17 import numpy as np
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 if HERE not in sys.path:
22     sys.path.append(HERE)
23 from style import apply_style, COLORS, MOD_COLORS, LABEL_COLORS # noqa: E402
24 from xai_common import MODALITIES, save_json, ROOT # noqa: E402
25
26 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
27
28
29 def load_all(out_dir, name):
30     fused = np.load(os.path.join(out_dir, "q3_temporal_fused_%s.npz" % name), allow_pickle=
31                     True)
32     shap = np.load(os.path.join(out_dir, "q3_shapley_%s.npz" % name), allow_pickle=True)
33     ev = json.load(open(os.path.join(out_dir, "q3_evidence_%s.json" % name), encoding="utf-8")
34                    )
35     return fused, shap, ev
36
37
38 def card_figure(sid, W, valid, rec, pi, score, prob, theta, energy, out_png,
39                label_cn="", meta=None):
40     plt = apply_style()
41     fig = plt.figure(figsize=(9.2, 6.4))
42     gs = fig.add_gridspec(3, 2, height_ratios=[1.0, 0.75, 1.15], hspace=0.55, wspace
43                        =0.22)
44
45     # —— (1) 三模态时间重要性条带 ——
46     ax1 = fig.add_subplot(gs[0, :])
47     for k, m in enumerate(MODALITIES):
48         w = np.asarray(W[k], np.float64)
49         n = int(valid[k].sum())
50         ax1.bar(np.arange(len(w)) + (k - 1) * 0.0, w, width=0.62, bottom=k, color=
51                MOD_COLORS[k],
52                label="%s (pi=%.2f)" % (MOD_CN[m], pi[k]))
53         ax1.axvline(n - 0.5, color="#666666", lw=0.8, ls=":")

```

```

50     top = np.argsort(-w[:n])[:3]
51     for p in top:
52         ax1.text(p, k + w[p] + 0.06, "*", ha="center", va="bottom",
53                 fontsize=6.5, color="#333333")
54 ax1.set_yticks([0.5, 1.5, 2.5])
55 ax1.set_yticklabels([MOD_CN[m] for m in MODALITIES])
56 ax1.set_xlabel("对齐段号 $k$ (文本为词元位置; 虚线为该模态有效长度上界)")
57 ax1.set_title("(a) 三模态时间重要性 $w^m_k$ (*=top-3 关键片段)", loc="left")
58 ax1.legend(loc="upper right", ncol=3)
59
60 # —— (2) 文本词着色 ——
61 ax2 = fig.add_subplot(gs[1, :])
62 ax2.axis("off")
63 te = rec.get("text_evidence", {})
64 words = te.get("top_words", [])
65 phrase = te.get("phrase", "")
66 span = te.get("phrase_char_span")
67 raw = (meta or {}).get("raw_text", "")
68 ax2.set_title("(b) 文本证据: 高亮为模型关注词, 方框为可回看的字符区间", loc="left")
69 txt = raw if raw else phrase
70 if txt:
71     hi = {w["word"].lower() for w in words if w.get("word")}
72     # 用等宽示意图: 把原句逐词着色 (步长按字符数估算, 换行留出边距)
73     import re
74     x, y = 0.0, 0.74
75     for tok in re.finditer(r"\S+", txt):
76         word = tok.group(0)
77         key = re.sub(r"^[A-Za-z0-9]", "", word).lower()
78         hot = key in hi
79         ax2.text(x, y, word, fontsize=9.5, va="center",
80                 color=(COLORS["accent"] if hot else "#333333"),
81                 weight=("bold" if hot else "normal"),
82                 bbox=(dict(facecolor="#FBEAD6", edgecolor="none", pad=1.2) if hot
83                        else None),
84                 transform=ax2.transAxes)
85         x += 0.0092 * (len(word) + 1.35)
86         if x > 0.995:
87             x, y = 0.0, y - 0.26
88     else:
89         ax2.text(0.01, 0.5, "(该样本无 raw_text, 无法回看原文)", transform=ax2.transAxes,
90                 fontsize=9)
91 if span:
92     ax2.text(0.0, 0.06, "字符区间 [%d,%d]: %s" % (span[0], span[1], phrase),
93             transform=ax2.transAxes, fontsize=8.5, color=COLORS["accent"])
94
95 # —— (3) 关键帧 + 音频能量 ——

```

```

95 ax3 = fig.add_subplot(gs[2, 0])
96 kf = (rec.get("vision_evidence") or {}).get("keyframe_file")
97 if kf and os.path.isfile(os.path.join(ROOT, kf)):
98     img = plt.imread(os.path.join(ROOT, kf))
99     ax3.imshow(img)
100     ax3.set_title("(c) 视觉关键帧 t=%0.2fs" % (rec["vision_evidence"].get("keyframe_time_s") or
101     0.0),
102     loc="left")
103 else:
104     ax3.text(0.5, 0.5, "无关键帧", ha="center", transform=ax3.transAxes)
105     ax3.set_title("(c) 视觉关键帧", loc="left")
106 ax3.set_xticks([])
107 ax3.set_yticks([])
108 ax3.grid(False)
109
110 ax4 = fig.add_subplot(gs[2, 1])
111 if energy is not None:
112     t, rms = energy["t"], energy["rms"]
113     ax4.plot(t, rms, color=COLORS["audio"], lw=1.2)
114     ae = rec.get("audio_evidence") or {}
115     if ae.get("start_s") is not None:
116         ax4.axvspan(ae["start_s"], ae["end_s"], color=COLORS["audio"], alpha=0.18)
117     ax4.set_xlabel("时间 / s")
118     ax4.set_ylabel("音频能量")
119     ax4.set_title("(d) 音频能量与证据时段（阴影）", loc="left")
120 else:
121     ax4.text(0.5, 0.5, "无音频能量", ha="center", transform=ax4.transAxes)
122     ax4.set_title("(d) 音频能量", loc="left")
123
124 sup = "预测: %s (强度 %+.3f, theta=%0.3f, 主模态 %s)" % (
125     label_cn, score, theta, MOD_CN[MODALITIES[int(np.argmax(pi))]])
126 fig.suptitle("解释卡. 样本 %s %s" % (sid, sup), fontsize=11.5, y=0.985)
127 os.makedirs(os.path.dirname(out_png), exist_ok=True)
128 fig.savefig(out_png, dpi=300)
129 plt.close(fig)
130 return out_png
131
132 def main():
133     ap = argparse.ArgumentParser()
134     ap.add_argument("--name", default="att4")
135     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
136     ap.add_argument("--fig_dir", default=os.path.join(ROOT, "figs"))
137     ap.add_argument("--theta", type=float, default=0.325)
138     ap.add_argument("--ids", default=None, help="逗号分隔; 默认自动挑（正/中/负各一）")
139     a = ap.parse_args()

```

```

140
141 fused, shap, ev = load_all(a.out_dir, a.name)
142 W, valid = fused["w_fused"], fused["valid"]
143 ids = [str(x) for x in fused["ids"]]
144 pi_all = shap["pi_soft"]
145 score = shap["score"]
146 prob = shap["prob"]
147 LAB = ("Negative", "Neutral", "Positive")
148 CN = {"Negative": "负向", "Neutral": "中性", "Positive": "正向"}
149 pred = [LAB[i] for i in prob.argmax(1)]
150 meta = {r["id"]: r for r in
151         json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
152                        encoding="utf-8"))[a.name]}
153
154 if a.ids:
155     pick = [s.strip() for s in a.ids.split(",")]
156 else: # 自动挑: 正/中/负各一条 (优先解释指标最有说服力的: 置信度高 + pi 集中)
157     pick = []
158     for lab in ("Positive", "Neutral", "Negative"):
159         cand = [i for i, p in enumerate(pred) if p == lab]
160         if not cand:
161             continue
162         best = max(cand, key=lambda i: (float(prob[i].max()) * float(np.max(pi_all[i
163                                     ]))
164                                     - np.min(pi_all
165                                     [i])))
166         pick.append(ids[best])
167     print("[PICK] %s" % pick)
168
169 made = []
170 cards = {}
171 for sid in pick:
172     i = ids.index(sid)
173     energy = None
174     ep = os.path.join(a.out_dir, "_audio", "%s_energy.npz" % sid)
175     if os.path.isfile(ep):
176         with np.load(ep) as z:
177             energy = dict(t=z["t"], rms=z["rms"])
178     card = dict(id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
179               prob={LAB[k]: round(float(prob[i, k]), 4) for k in range(3)},
180               confidence=round(float(prob[i].max()), 4), theta=a.theta,
181               main_modality=MODALITIES[int(np.argmax(pi_all[i]))],
182               modality_contribution={MODALITIES[k]: round(float(pi_all[i, k]), 4)
183                                   for k in range(3)},
184               shapley_raw={MODALITIES[k]: round(float(shap["phi_mu_miss"][i, k]), 4)
185                           for k in range(3)},

```

```

184         temporal_importance={MODALITIES[k]: np.round(W[i, k], 4).tolist()
185                               for k in range(3)},
186         evidence=ev.get(sid, {}))
187     png = os.path.join(a.fig_dir, "Q3_解释卡_%s_%s_组合图.png" % (sid, pred[i]))
188     card_figure(sid, W[i], valid[i], ev.get(sid, {}), pi_all[i], float(score[i]),
189               prob[i], a.theta, energy, png, CN[pred[i]], meta.get(sid, {}))
190     cards[sid] = card
191     made.append(os.path.relpath(png, ROOT))
192     print(" [CARD] %s -> %s" % (sid, os.path.relpath(png, ROOT)))
193
194     save_json(os.path.join(a.out_dir, "q3_cards_%s.json" % a.name), cards)
195     print("[SAVE] %d 张解释卡: %s" % (len(made), made))
196     print("CARD_OK")
197     return 0
198
199
200 if __name__ == "__main__":
201     sys.exit(main())

```

C.12 问题三：附件四全量推理与解释输出

code/03_xai/infer_explain_att4.py (共 230 行)

```

1  # -*- coding: utf-8 -*-
2  r"""S6-b: 主入口 —— 生成附件4 的”预测 + 解释”提交文件与解释卡。
3
4  产物:
5      submission/pred_explain_att4.csv    题目要求的附件4 预测与解释结果（每行一条样本）
6      submission/explain_cards/{id}.json  逐样本完整解释卡（含 50 步时间重要性）
7      submission/evidence/{id}_kf.jpg    关键帧（由 evidence.py 生成）
8
9  内建自查断言（任一不过即报错退出）:
10     1. 行数与 id 顺序与附件4 严格一致;
11     2. pred_label / pred_score 与 submission/pred_att4.csv 逐位一致（同一骨干、同一 theta）;
12     3. 解释数值有限、pi 和为 1、时间重要性非负;
13     4. 证据区间合法（0 <= start < end <= 有效长度; 秒 <= 视频时长）。
14
15  用法:
16     python infer_explain_att4.py --out_dir ../../runs/q3 --sub_dir ../../submission
17     """
18  import argparse
19  import json
20  import os
21  import re
22  import subprocess
23  import sys

```

```

24
25 import numpy as np
26 import pandas as pd
27 import torch
28
29 HERE = os.path.dirname(os.path.abspath(__file__))
30 if HERE not in sys.path:
31     sys.path.append(HERE)
32 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa:
33     E402
34     predict_batch, save_json, q3_meta, ROOT)
35 LABELS = ("Negative", "Neutral", "Positive")
36 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
37
38
39 def probe_fps(video, cache_path):
40     """用 ffmpeg 的 stderr 解析帧率（缓存复用）。"""
41     cache = {}
42     if os.path.isfile(cache_path):
43         try:
44             cache = json.load(open(cache_path, encoding="utf-8"))
45         except Exception:
46             cache = {}
47     key = os.path.basename(video)
48     if key in cache:
49         return cache[key]
50     import imageio_ffmpeg
51     r = subprocess.run([imageio_ffmpeg.get_ffmpeg_exe(), "-i", video],
52         capture_output=True)
53     txt = (r.stderr or b"").decode("utf-8", "replace")
54     m = re.search(r"(\d+(?:\.\d+)?)\s*fps", txt)
55     fps = float(m.group(1)) if m else None
56     cache[key] = fps
57     try:
58         os.makedirs(os.path.dirname(cache_path), exist_ok=True)
59         json.dump(cache, open(cache_path, "w", encoding="utf-8"), ensure_ascii=False,
60             indent=1)
61     except OSError:
62         pass
63     return fps
64
65 def main():
66     ap = argparse.ArgumentParser()
67     ap.add_argument("--pkI", default=os.path.join(ROOT, "data_att", "att4_aligned.pkI"))

```

```

68 ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
69 ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
70 ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
71 ap.add_argument("--theta", type=float, default=0.325)
72 ap.add_argument("--device", default="cpu")
73 a = ap.parse_args()
74
75 models, stats, theta_ck = load_backbone(a.ckpt_dir, a.device, verbose=False)
76 split, key = load_split(a.pkl, stats)
77 N = split["N"]
78 ids = [str(x) for x in split["ids"]]
79 print("[DATA] att4 N=%d (ckpt theta 均值 %.3f, 提交用 theta=%.3f)" % (N, theta_ck, a.theta))
80
81 # ----- 1) 复现预测 -----
82 b = make_batch(split, list(range(N)), a.device)
83 out = predict_batch(models, b, target="mu")
84 score = out["score"].detach().cpu().numpy()
85 prob = out["prob"].detach().cpu().numpy()
86 pred = [LABELS[i] for i in prob.argmax(1)]
87 ref_path = os.path.join(a.sub_dir, "pred_att4.csv")
88 assert os.path.isfile(ref_path), "缺少 %s (先跑 dl_recalib.py)" % ref_path
89 ref = pd.read_csv(ref_path, dtype={"id": str})
90 ref_ids = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in ref["id"]]
91 assert ref_ids == ids, "附件4 id 顺序不一致: %s" % ref_ids[:4]
92 lab_eq = float((np.array(pred) == ref["pred_label"].values).mean())
93 sc_diff = float(np.abs(score - ref["pred_score"].values).max())
94 print("[CHECK] 与 pred_att4.csv: 标签一致率 %.3f; 分数最大差 %.1e" % (lab_eq, sc_diff))
95 assert lab_eq == 1.0, "标签与 pred_att4.csv 不一致, 禁止输出"
96 assert sc_diff < 1e-3, "分数与 pred_att4.csv 不一致 (差 %.4f)" % sc_diff
97
98 # ----- 2) 读解释与证据 -----
99 fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_att4.npz"), allow_pickle=True)
100 shap = np.load(os.path.join(a.out_dir, "q3_shapley_att4.npz"), allow_pickle=True)
101 met = np.load(os.path.join(a.out_dir, "q3_temporal_occ_att4.npz"), allow_pickle=True)
102 ev = json.load(open(os.path.join(a.out_dir, "q3_evidence_att4.json"), encoding="utf-8"))
103 faith_p = os.path.join(a.out_dir, "q3_faithfulness_att4.csv")
104 faith = pd.read_csv(faith_p, dtype={"id": str}) if os.path.isfile(faith_p) else None
105 meta = {r["id"]: r for r in json.load(
106     open(os.path.join(ROOT, "data_att", "att_meta.json"), encoding="utf-8"))["att4"]}
107
108 W = fused["w_fused"]
109 W_att, W_ig, W_occ = fused["w_att"], fused["w_ig"], fused["w_occ"]
110 valid = fused["valid"]
111 pi = shap["pi_soft"]
112 pi_relu = shap["pi_relu"]
113 phi = shap["phi_mu_miss"]

```

```

114 occ = met["w_occ"]
115 fps_cache = os.path.join(a.out_dir, "video_fps.json")
116
117 rows, cards = [], {}
118 for i, sid in enumerate(ids):
119     n_text = int(valid[i, 0].sum())
120     n_audio = int(valid[i, 1].sum())
121     n_vision = int(valid[i, 2].sum())
122     e = ev.get(sid, {})
123     m = meta.get(sid, {})
124     # 断言: pi 归一化与时间重要性合法性
125     assert abs(float(pi[i].sum()) - 1.0) < 1e-6, "pi 未归一化: %s" % sid
126     assert np.isfinite(W[i]).all() and (W[i] >= -1e-9).all(), "时间重要性非法: %s" %
sid
127
128     te = e.get("text_evidence", {}) or {}
129     tr = te.get("seg_runs") or [[None, None]]
130     ae = e.get("audio_evidence", {}) or {}
131     ar = ae.get("seg_runs") or [[None, None]]
132     ve = e.get("vision_evidence", {}) or {}
133     vr = ve.get("seg_runs") or [[None, None]]
134     kf_t = ve.get("keyframe_time_s")
135     fps = probe_fps(m["video"], fps_cache) if m.get("video") else None
136     frame_idx = int(round(kf_t * fps)) if (kf_t is not None and fps) else None
137
138     # 证据合法性断言
139     if tr and tr[0][0] is not None:
140         assert 0 <= tr[0][0] < max(1, n_text), "文本证据段号非法: %s" % sid
141     if ae.get("end_s") is not None and m.get("duration_s"):
142         assert ae["end_s"] <= m["duration_s"] + 0.05, "语音证据超视频时长: %s" % sid
143
144     # 忠实性/充分性/稳定性/稀疏性 (来自 metrics_xai.py, 取融合解释那一行)
145     compr = suff = stab = spar = None
146     if faith is not None:
147         sub = faith[(faith["id"] == sid) & (faith["source"] == "fused")]
148         if len(sub):
149             compr = float(sub["compr_mean"].iloc[0])
150             suff = float(sub["suff_absmean"].iloc[0])
151             spar = float(sub["sparsity_mean"].iloc[0])
152     rep = json.load(open(os.path.join(a.out_dir, "q3_faithfulness_att4_report.json"),
153                          encoding="utf-8")) if os.path.isfile(
154         os.path.join(a.out_dir, "q3_faithfulness_att4_report.json")) else {}
155     stab = (rep.get("stability", {}).get(sid, {}) or {}).get("stability_fused")
156
157     phrase = te.get("phrase", "")
158     row = dict(

```

```

159     id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
160     prob_neg=round(float(prob[i, 0]), 4), prob_neu=round(float(prob[i, 1]), 4),
161     prob_pos=round(float(prob[i, 2]), 4),
162     confidence=round(float(prob[i].max()), 4), theta=a.theta,
163     main_modality=MODALITIES[int(np.argmax(pi[i]))],
164     w_text=round(float(pi[i, 0]), 4), w_audio=round(float(pi[i, 1]), 4),
165     w_vision=round(float(pi[i, 2]), 4),
166     w_text_relu=round(float(pi_relu[i, 0]), 4),
167     w_audio_relu=round(float(pi_relu[i, 1]), 4),
168     w_vision_relu=round(float(pi_relu[i, 2]), 4),
169     phi_text=round(float(phi[i, 0]), 4), phi_audio=round(float(phi[i, 1]), 4),
170     phi_vision=round(float(phi[i, 2]), 4),
171     text_evidence=phrase or te.get("note", ""),
172     text_seg_start=tr[0][0], text_seg_end=tr[0][1],
173     text_char_start=(te.get("phrase_char_span") or [None, None])[0],
174     text_char_end=(te.get("phrase_char_span") or [None, None])[1],
175     audio_start_s=ae.get("start_s"), audio_end_s=ae.get("end_s"),
176     audio_seg_start=ar[0][0], audio_seg_end=ar[0][1],
177     vision_seg_start=vr[0][0], vision_seg_end=vr[0][1],
178     vision_frame_idx=frame_idx, vision_time_s=kf_t,
179     vision_keyframe=ve.get("keyframe_file", ""),
180     compr=None if compr is None else round(compr, 4),
181     suff=None if suff is None else round(suff, 4),
182     stability=stab, sparsity=None if spar is None else round(spar, 4),
183 )
184 rows.append(row)
185 cards[sid] = dict(
186     id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
187     prob={LABELS[k]: round(float(prob[i, k]), 4) for k in range(3)},
188     confidence=round(float(prob[i].max()), 4), theta=a.theta,
189     main_modality=row["main_modality"],
190     modality_contribution={MODALITIES[k]: round(float(pi[i, k]), 4) for k in
range(3)},
191     modality_contribution_relu={MODALITIES[k]: round(float(pi_relu[i, k]), 4)
for k in range(3)},
192     shapley_raw={MODALITIES[k]: round(float(phi[i, k]), 4) for k in range(3)},
193     temporal_importance_fused={MODALITIES[k]: np.round(W[i, k], 4).tolist()
for k in range(3)},
194     temporal_importance_att={MODALITIES[k]: np.round(W_att[i, k], 4).tolist()
for k in range(3)},
195     temporal_importance_ig={MODALITIES[k]: np.round(W_ig[i, k], 4).tolist()
for k in range(3)},
196     temporal_importance_occ={MODALITIES[k]: np.round(W_occ[i, k], 4).tolist()
for k in range(3)},
197     gamma=fused["gamma"].tolist(),
198     evidence=dict(text=te, audio=ae, vision=ve),
199

```

```

204         raw_text=m.get("raw_text", ""), video=os.path.basename(m.get("video", "") or ""),
205         duration_s=m.get("duration_s"),
206         faithfulness=dict(comprehensiveness=row["compr"], sufficiency=row["suff"],
207                           stability=stab, sparsity=row["sparsity"]))
208
209     df = pd.DataFrame(rows)
210     out_csv = os.path.join(a.sub_dir, "pred_explain_att4.csv")
211     df.to_csv(out_csv, index=False, encoding="utf-8-sig")
212     os.makedirs(os.path.join(a.sub_dir, "explain_cards"), exist_ok=True)
213     for sid, c in cards.items():
214         with open(os.path.join(a.sub_dir, "explain_cards", "%s.json" % sid), "w",
215                  encoding="utf-8") as f:
216             json.dump(c, f, ensure_ascii=False, indent=1)
217     save_json(os.path.join(a.out_dir, "q3_explain_meta.json"),
218              dict(meta=q3_meta(a.ckpt_dir, models, a.theta, dict(pk1=os.path.relpath(a.
219                                pk1, ROOT))),
220                  csv=os.path.relpath(out_csv, ROOT), n=N,
221                  columns=list(df.columns)))
222     print("[SAVE] %s (%d 行 x %d 列)" % (os.path.relpath(out_csv, ROOT), len(df), df.shape
223                                         [1]))
224     print("[SAVE] submission/explain_cards/*.json (%d 个)" % len(cards))
225     cnt = df["main_modality"].value_counts().to_dict()
226     print("[STAT] 主参考模态分布: %s" % cnt)
227     print("INFER_EXPLAIN_OK")
228     return 0
229
230 if __name__ == "__main__":
231     sys.exit(main())

```

C.13 问题三：交付自查（17 项一致性校验）

code/03_xai/verify_q3.py（共 150 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S9: 问题3 交付自查（任一断言不过即失败）。
3
4  检查项：
5  1. pred_explain_att4.csv 行数 = 20、id 顺序与附件4 文件名一致、列齐全；
6  2. 其 pred_label / pred_score 与 submission/pred_att4.csv 逐位一致；
7  3. 每行模态作用度之和为 1；时间重要性数值合法（在卡片 JSON 中抽检）；
8  4. 证据区间合法（段号、秒区间不超过视频时长）；
9  5. 关键帧文件存在且非空；
10 6. 解释卡 JSON 每个 id 齐全；
11 7. submission/ 总体积 <= 50 MB；
12 8. 匿名性：CSV/JSON 中不含单位/队号/学号/手机号/邮箱等身份信息。

```

```

13
14 用法:
15 python verify_q3.py --sub_dir ../\submission --out_dir ../\runs\q3
16 """
17 import argparse
18 import json
19 import os
20 import re
21 import sys
22
23 import numpy as np
24 import pandas as pd
25
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 if HERE not in sys.path:
28     sys.path.append(HERE)
29 from xai_common import save_json, ROOT          # noqa: E402
30
31 BANNED = [r"参赛队", r"队号", r"学号", r"指导教[师]?", r"大[学]?[\u4e00-\u9fff]{0,6}学院",
32           r"1[3-9]\d{9}", r"[\w.\-]+@[ \w.\-]+\.\w+", r"[\u4e00-\u9fff]{2,4}(同学|老师)"]
33
34
35 def main():
36     ap = argparse.ArgumentParser()
37     ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
38     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
39     ap.add_argument("--max_mb", type=float, default=50.0)
40     a = ap.parse_args()
41     checks, fails = [], []
42
43     def ck(name, ok, detail=""):
44         checks.append(dict(check=name, ok=bool(ok), detail=str(detail)))
45         if not ok:
46             fails.append("%s: %s" % (name, detail))
47         print(" [%s] %-34s %s" % ("OK" if ok else "FAIL", name, detail))
48
49     # 1) 行数 / id / 列
50     path = os.path.join(a.sub_dir, "pred_explain_att4.csv")
51     ck("文件存在", os.path.isfile(path), os.path.relpath(path, ROOT))
52     df = pd.read_csv(path, dtype={"id": str})
53     df["id"] = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in df["id"]]
54     meta4 = json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
55                           encoding="utf-8"))
56     ids = [r["id"] for r in meta4["att4"]]
57     ck("行数=20", len(df) == 20, "实为 %d" % len(df))
58     ck("id 顺序与附件4 一致", list(df["id"]) == ids, "%s" % list(df["id"])[ :4])

```

```

59 need = ["id", "pred_label", "pred_score", "prob_neg", "prob_neu", "prob_pos",
60         "main_modality", "w_text", "w_audio", "w_vision", "text_evidence",
61         "text_seg_start", "text_seg_end", "audio_start_s", "audio_end_s",
62         "vision_frame_idx", "vision_time_s", "compr", "suff", "stability", "sparsity"]
63 miss = [c for c in need if c not in df.columns]
64 ck("必需列齐全", not miss, "缺%s" % miss)
65
66 # 2) 与 pred_att4.csv 一致
67 ref = pd.read_csv(os.path.join(a.sub_dir, "pred_att4.csv"), dtype={"id": str})
68 ck("标签与 pred_att4.csv 一致",
69     float((df["pred_label"].values == ref["pred_label"].values).mean()) == 1.0)
70 ck("分数与 pred_att4.csv 一致",
71     float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()) < 1e-3,
72     "最大差 %2e" % float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()))
73
74 # 3) pi 归一化
75 s = df[["w_text", "w_audio", "w_vision"]].values.sum(1)
76 ck("模态作用度之和=1", float(np.abs(s - 1).max()) < 2e-3, "最大偏差 %2e"
77     % float(np.abs(s - 1).max()))
78 ck("主模态取值合法",
79     set(df["main_modality"]) <= {"text", "audio", "vision"},
80     "%s" % sorted(set(df["main_modality"])))
81
82 # 4) 证据合法性
83 bad_seg = df[(df["text_seg_start"].notna()) & (df["text_seg_end"].notna()) &
84              (~(df["text_seg_start"] >= 0) & (df["text_seg_end"] <= 50) &
85               (df["text_seg_start"] < df["text_seg_end"])))])
86 ck("文本段号合法", len(bad_seg) == 0, "越界 %d 行" % len(bad_seg))
87 dur = {r["id"]: r.get("duration_s") for r in meta4["att4"]}
88 bad_t = [i for i, r in df.iterrows()
89           if pd.notna(r["audio_end_s"]) and dur.get(r["id"]) and
90           r["audio_end_s"] > dur[r["id"]] + 0.05]
91 ck("语音时段不超视频时长", len(bad_t) == 0, "越界 %d 行" % len(bad_t))
92 ck("含文本证据的样本数>0", int(df["text_evidence"].notna().sum()) > 0,
93     "%d/20" % int(df["text_evidence"].notna().sum()))
94
95 # 5) 关键帧
96 kf = [os.path.join(ROOT, p) for p in df["vision_keyframe"].dropna().astype(str) if p]
97 ok_kf = [p for p in kf if os.path.isfile(p) and os.path.getsize(p) > 500]
98 ck("关键帧文件有效", len(ok_kf) >= max(1, int(0.8 * len(kf))),
99     "%d/%d 张可用" % (len(ok_kf), len(kf)))
100
101 # 6) 解释卡
102 cdir = os.path.join(a.sub_dir, "explain_cards")
103 have = sorted(os.path.splitext(f)[0] for f in os.listdir(cdir) if os.path.isdir(cdir)
104               ) else []

```

```

104 ck("解释卡 JSON 齐全", set(have) == set(ids), "%d/20" % len(have))
105 if set(have) >= set(ids):
106     c = json.load(open(os.path.join(cdir, ids[0] + ".json"), encoding="utf-8"))
107     ti = c["temporal_importance_fused"]["text"]
108     ck("时间重要性长度=50", len(ti) == 50, "len=%d" % len(ti))
109     ck("时间重要性非负且有界", bool(np.min(ti) >= -1e-6 and np.max(ti) <= 1 + 1e-6))
110
111 # 7) 体积
112 tot = 0
113 for r, _, fs in os.walk(a.sub_dir):
114     if "_backup" in r:
115         continue
116     for f in fs:
117         tot += os.path.getsize(os.path.join(r, f))
118 mb = tot / 1024 / 1024
119 ck("submission 体积 <= %.0fMB" % a.max_mb, mb <= a.max_mb, "%.2fMB" % mb)
120
121 # 8) 匿名性
122 hits = []
123 for r, _, fs in os.walk(a.sub_dir):
124     if "_backup" in r:
125         continue
126     for f in fs:
127         if not f.lower().endswith((".csv", ".json", ".md", ".txt")):
128             continue
129         p = os.path.join(r, f)
130         try:
131             txt = open(p, encoding="utf-8", errors="replace").read()
132         except OSError:
133             continue
134         for pat in BANNED:
135             for m in re.finditer(pat, txt):
136                 hits.append("%s :: %s" % (os.path.relpath(p, ROOT), m.group(0)))
137 ck("无身份信息", not hits, "%d 处命中" % len(hits))
138 if hits:
139     print("    %s" % hits[:5])
140
141 rep = dict(n_check=len(checks), n_fail=len(fails), checks=checks, fails=fails,
142           submission_mb=round(mb, 2))
143 save_json(os.path.join(a.out_dir, "verify_q3.json"), rep)
144 print("\n%s (%d 项检查, %d 项失败)" % ("VERIFY_PASS" if not fails else "VERIFY_FAIL",
145                                     len(checks), len(fails)))
146 return 0 if not fails else 1
147
148
149 if __name__ == "__main__":

```

```
150 sys.exit(main())
```